

Living Map System Architecture

Writer robot · LMB2 beacon network · three gateways ·
Outside Network Area · Command Post · Executor robot

Contents

1. The system in one page	4
1.1 The chain of a mission, step by step	4
1.2 The pieces and where their code lives	5
1.3 Simulation and real hardware	5
2. Deployment: where everything runs	6
3. Robots, worlds and localisation	7
3.1 The robot model	7
3.2 Worlds	7
3.3 Frames and localisation	7
4. Writer software	8
4.1 Topics	9
4.2 What run_explorer.sh starts	10
5. The explorer brain (explorer_core.py)	10
5.1 Planning grid	10
5.2 Targets	10
5.3 Path following	11
5.4 Hazards, keep-out zones and the retreat	11
5.5 States	12
5.6 Safety net (always on)	12
5.7 Parameters (frontier_explorer)	13
6. Vision: seeing the hazard blocks	14
6.1 Pipeline, per colour frame	14
6.2 Health log	15
6.3 Parameters (vision_event_detector)	15
7. Beacons: dropping, linking, recording	15
7.1 When a beacon is dropped (beacon_drop_node + beacon_logic.py)	15
7.2 How beacons are linked: the tree	16
7.3 The /writer/beacon_dropped message	16
7.4 From a beacon to a signed record (mission_recorder + mission_log.py)	16
8. The LMB2 beacon network	17
8.1 The 44-byte frame	17
8.2 Kinds and adaptive aging	18
8.3 Security	18
8.4 Radio	19
8.5 The gossip engine	19
8.6 What happens when things break	20

8.7 Provisioning a real beacon	20
8.8 Hardware	20
8.9 Verification of the network	20
9. Gateways and the Command Post	21
9.1 The gateway board	21
9.2 One gateway without the ONA: gateway_bridge.py	21
9.3 The Command Post dashboard	21
9.4 Other ways into the dashboard	22
10. The Outside Network Area	23
10.1 Three gateways that vote	23
10.2 Where is the robot? Three spheres	24
10.3 GPS for the Command Post: frame translation	25
10.4 Store and forward: LTE, satellite, nothing lost	25
10.5 Briefing the Executor	26
10.6 The new frames	26
10.7 The robots in Gazebo: ona_link	26
10.8 What the operator sees	27
10.9 From the GALERIA code	28
11. The Executor robot	28
11.1 Receiving the beacons	29
11.2 Rebuilding the tree	29
11.3 Choosing targets	29
11.4 Phases	30
11.5 Briefed by the Command Post	30
11.6 Topics and parameters	31
11.7 Beacon-graph navigation (ready, not wired in yet)	32
12. Simulation and verification	32
13. Data contracts	34
14. The Gafsa mine scenario	35
15. Limitations and next steps	36

1. The system in one page

The Living Map is a two-robot rescue chain for a contaminated or collapsed building, connected to a distant Command Post. A **Writer** robot goes in first, maps the building and leaves a trail of small LoRa radio beacons. Each beacon carries one signed 44-byte record: what is here (exit, trail point, radiation, fire, gas, victim), where, how sure, and which beacon is the next one on the way out. The beacons copy each other's records over the radio (gossip), so the whole map lives in every beacon. Outside the building **three gateways** listen. The **Outside Network Area (ONA)** accepts a record only when two gateways heard the identical signed bytes, locates the robots from the gateways' distance measurements (like GPS), and forwards everything to the **Command Post** dashboard over LTE, or over satellite when LTE fails. The **Executor** robot then enters knowing nothing but the radio frames. Briefed from the Command Post, it drives beacon by beacon to its targets, treats them, and follows the beacons back out.

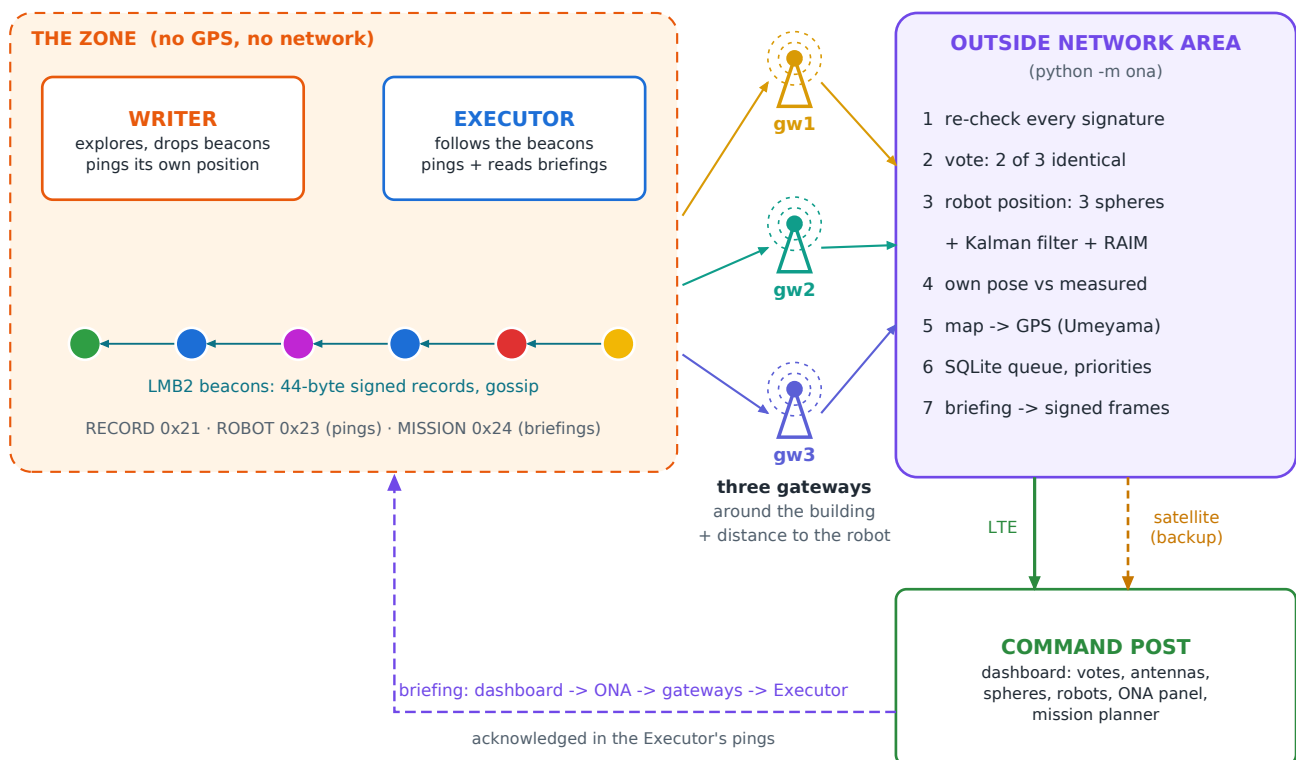


Figure 1. The whole chain. Everything between the zone and the Command Post passes through the ONA; briefings go back in through the same three gateways.

1.1 The chain of a mission, step by step

- 1 Writer explores.** SLAM builds the map; the frontier explorer drives to unexplored edges and points the camera at every area. A hazard seen by the camera becomes a 1.2 m keep-out zone; if it blocks the way the robot turns back along its own trail and takes another route (section 5).
- 2 Writer drops beacons.** An EXIT beacon at the entrance, trail beacons at turns and every 2.5 m, and a hazard beacon where the camera reported each hazard. Every new beacon points to the last beacon dropped or passed, so all beacons form a tree rooted at the EXIT (section 7).
- 3 Each beacon becomes an LMB2 record.** Position in lat/lon, error estimate, kind, severity, confidence with a half-life, the next hop (id, distance, compass bearing), time-to-live, all sealed with

an HMAC. In simulation the `mission_recorder` writes them to `~/writer_robot_ws/missions/latest.json` (section 7.4).

- 4 **Beacons gossip.** Own announce every round, flood relays, background repair, pull digests, lost-beacon detection, all within the 1 % radio duty cycle (section 8).
- 5 **Three gateways → the ONA.** It re-checks every signature, accepts a record when two gateways agree, names a gateway that lies, locates each robot from three distances, compares that with where the robot thinks it is, and queues everything for the Command Post (section 10).
- 6 **Command Post.** The dashboard draws fading pins, the way-out routes, lost beacons, the gateways and their votes, and the robots on their distance circles. The operator picks the Executor's targets and dispatches them (section 9.3).
- 7 **Briefing.** The ONA signs the mission as MISSION frames, the gateways transmit it, and the Executor acknowledges it in its position reports (section 10.5).
- 8 **Executor completes the mission.** It verifies the frames, rebuilds the beacon tree, drives along it to the briefed targets in order (every live hazard without a briefing), treats each from 1.6–2.8 m, and returns to the EXIT (section 11).

1.2 The pieces and where their code lives

The whole project is one folder, `living_map`, with four parts: `writer_robot`, `beacon_net`, `ona` and `command_post`.

component	runs on	code	section
Writer robot (simulated)	Ubuntu VM: ROS 2 Lyrical + Gazebo Jetty	<code>writer_robot/</code> (installed as <code>~/writer_robot_ws/src/writer_robot</code>)	3–7
Executor robot (simulated)	VM, same package	<code>executor_core.py</code> , <code>executor_node.py</code>	11
robot ↔ gateways radio (simulated)	VM, same package	<code>ona_link_node.py</code> , <code>ona_radio.py</code>	10.7
2D simulators and tests	VM or Windows (plain Python)	<code>writer_robot/tools/</code>	12
LMB2 protocol + gossip engine	ESP32 firmware, PC simulator, Python	<code>beacon_net/lib</code> , <code>firmware</code> , <code>python</code>	8
network simulator bpsim	VM (make) or Windows (<code>bpsim.exe</code>)	<code>beacon_net/host/bpsim.c</code>	8.9
gateway firmware, one-gateway bridge	ESP32; Python	<code>beacon_net/firmware/gateway_node</code> , <code>beacon_net/python/beaconnet/gateway_bridge.py</code>	9
Outside Network Area	VM or Windows (Python 3 + <code>numpy</code>)	<code>ona/</code> (<code>python -m ona</code>)	10
Command Post dashboard	Windows or VM (Node.js)	<code>command_post/</code>	9.3

1.3 Simulation and real hardware

Every link of the chain has a simulated stand-in, so the whole mission can be shown without hardware. The stand-ins speak the same formats as the real parts, so each can be swapped for the real thing on its own.

link	in simulation (today)	with real hardware
beacon drop	Gazebo spawns a static beacon model	trapdoor servo drops a physical beacon
record into the beacon	<code>mission_recorder</code> writes the signed frames to a mission file	<code>beacon_record_node</code> sends TIME, PROV, SEED over USB

link	in simulation (today)	with real hardware
beacon network	bpsim runs the real gossip code on a simulated 868 MHz channel	ESP32 + SX1276 beacons
three gateways	ona_link (robots in Gazebo), the zone simulator, or bpsim --gateways 3	three gateway boards on USB to the ONA computer (--serial)
robot distances	from the true Gazebo pose, through the world's walls (each wall its own loss and extra delay)	a time-of-flight ranging radio (SX1280 2.4 GHz, or UWB) on each gateway and robot
uplink	LTE up, outages on demand (--lte-outage), satellite modelled (Iridium SBD, 340 bytes)	LTE modem; Iridium SBD modem
radio → Executor	beacon_radio_sim replays the mission file; briefings through ona_link	a LoRa receiver on the Executor (bridge to /executor/lora_rx and /executor/briefing still to write)

2. Deployment: where everything runs

The robots need Linux (ROS 2 and Gazebo), so they run in the Ubuntu VirtualBox VM. Everything else also runs natively on Windows: the dashboard, the ONA, the beacon simulator (prebuilt .exe), the 2D robot simulators, the tests and the ESP32 firmware tools. The same `living_map` folder is unzipped in both places.

Copy `living_map.zip` into the VM (~Downloads or a shared folder) and unzip it on Windows too.

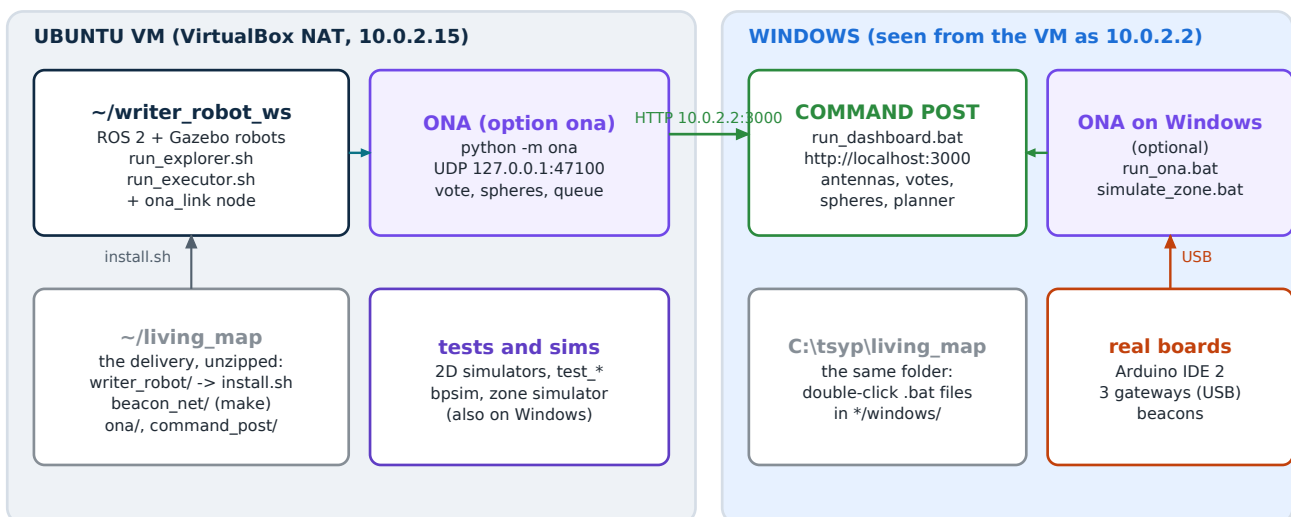


Figure 2. What runs in the VM and what runs on Windows.

place	what is there
~/living_map/	the delivery unzipped in the VM: <code>writer_robot/</code> , <code>beacon_net/</code> , <code>ona/</code> , <code>command_post/</code> , <code>docs/</code> , <code>START_HERE.md</code> , <code>COMMANDS.txt</code>
~/writer_robot_ws/	ROS workspace made by <code>writer_robot/install.sh</code> : <code>src/writer_robot</code> (the package and a copy of <code>ona/</code>), <code>build/</code> , <code>install/</code> , the scripts <code>run_explorer.sh</code> , <code>run_executor.sh</code> , <code>stop_demo.sh</code> , <code>check_sim.sh</code> , and <code>missions/</code> (recorded missions)
~/writer_robot_backups/	every package or script replaced by <code>install.sh</code> (never deleted)
C:\tsyp\living_map\	the same folder on Windows; <code><part>\windows*.bat</code> are the double-click launchers
/tmp/*.log (VM)	one log per process of a run (list in the Guide, part M)

NOTE VirtualBox NAT: the VM reaches the Windows host at **10.0.2.2**. A dashboard on Windows is therefore `http://10.0.2.2:3000` for programs in the VM and `http://localhost:3000` in the Windows browser. The ONA and `ona_link` talk over UDP port 47100 (inside the VM, or VM → Windows with `ONA_HOST=10.0.2.2`).

3. Robots, worlds and localisation

3.1 The robot model

Both robots share one URDF (`urdf/writer_robot.urdf.xacro`) with a `role` argument. The launch file `gazebo_sim.launch.py robot:=writer|executor` picks the role and names the model `writer_robot` or `executor_robot`.

part	Writer	Executor
base	4-wheel skid steer (Gazebo DiffDrive), body 0.43 × 0.38 m, orange	same base, blue
LiDAR	gpu_lidar, 180 samples, 8 Hz, on a 14 cm mast: scan plane 0.34 m above the floor	same
camera	RGB-D 320 × 240 at 6 Hz, horizontal FOV 1.089 rad (62°), 0.213 m ahead of the base, optical centre 0.135 m above the floor	same
IMU	50 Hz (its gyro yaw rate feeds the EKF of the ekf option)	same
payload	beacon magazine (120), trapdoor servo on <code>/trapdoor_cmd</code> (gz JointPositionController)	treatment canister (<code>tool_link</code>), no magazine

The LiDAR mast matters: at 0.20 m the scan plane would cut through the robot's own magazine and the dropped beacons and print phantom walls into the SLAM map. At 0.34 m the scan passes above the magazine, the antenna and every beacon (< 0.17 m); the explorer also ignores any LiDAR point inside the robot footprint.

3.2 Worlds

world file	content
<code>contaminated_zone.world</code> (default, "big")	20 × 14 m building: hall, 6 rooms, a loop of corridors around an island room, a zigzag east wing, 6 rubble piles. Hazards: 2 purple radiation blocks (one in the narrow north corridor, where the LiDAR sees a gap and only the camera says no), 1 red fire block, 1 yellow gas block.
<code>retreat_test.world</code> ("retreat")	small and quick: a purple block 3 m after a corner, so the camera sees it only after turning, and the robot must turn back.
<code>contaminated_zone_small.world</code> ("small")	the original small arena.
<code>/tmp/executor_world.world</code>	made by <code>mission_world</code> for each Executor run: the Writer's world plus a static beacon model at every drop point (white LED on the EXIT, blue on the others).

The layouts are generated by `tools/make_world.py` (edit the lists there and re-run it). The 2D simulators read the same `.world` files, so every test runs on the exact Gazebo geometry.

3.3 Frames and localisation

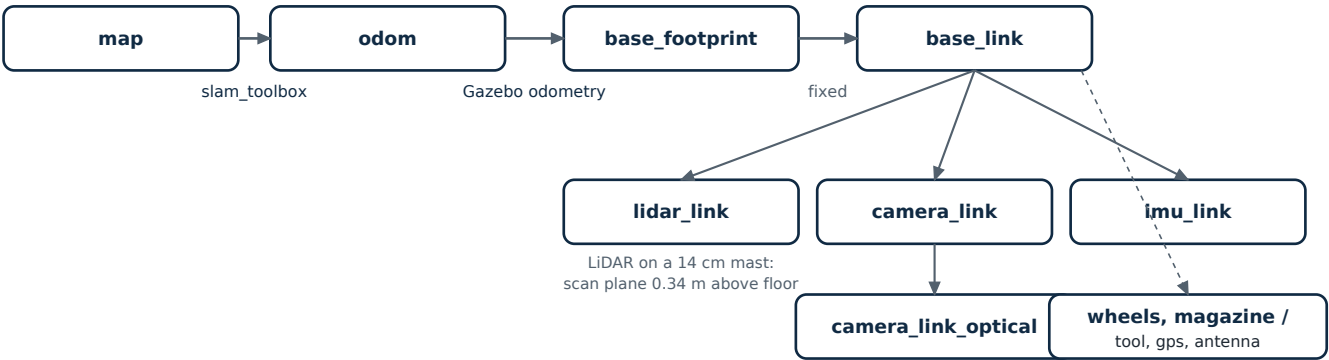


Figure 3. TF tree. `slam_toolbox` owns `map`→`odom`; Gazebo's odometry (or the EKF) owns `odom`→`base_footprint`.

Odometry. In the simulation, `odom`→`base_footprint` is the robot's true motion, published by Gazebo's `OdometryPublisher` system (in the URDF) on `/odom` and on `/odom_tf`, which the launch file bridges to `/tf`. An EKF of the wheel odometry and the IMU did not follow the turns of the 4-wheel skid-steer robot in Gazebo: in RViz the robot stood still while the map turned around it and `slam_toolbox` added no scans. The EKF is kept for experiments, and for the real robot (section 14): `run_explorer.sh ekf → odom_source:=ekf`, with `config/ekf.yaml` fusing the wheel speeds from `/wheel_odom` and the gyro yaw rate.

	default: odom_source gz	option ekf: odom_source ekf
odom→base_footprint	Gazebo OdometryPublisher, bridged /odom_tf → /tf	robot_localization EKF
/odom	true motion (OdometryPublisher)	true motion (unused by the EKF)
EKF inputs	-	/wheel_odom (vx, vy, yaw rate) + /imu (yaw rate)
SLAM file	slam_toolbox_params.yaml: scan matching off (nothing to correct)	slam_toolbox_params_ekf.yaml: scan matching + loop closing on

SLAM. Asynchronous `slam_toolbox`, `map` published every 2 s, a scan added after 0.3 m or 0.3 rad of motion, 0.5 s transform timeout (Gazebo in a VM runs slower than real time), 30 s TF buffer. The scripts wait for the odometry TF, bring the lifecycle node up with `configure` then `activate`, and wait for `map`→`odom` before starting the robot brain. `bash ~/writer_robot_ws/check_sim.sh` prints every rate and TF of a running simulation.

The map frame and GPS. Each robot's map frame starts where it starts (the EXIT). The anchor of the mission (lat/lon of that point and the compass direction of map +x) turns map x/y into lat/lon for the records (section 7.4); the ONA uses the same anchor for the robots' positions and re-checks it during the mission (section 10.3).

4. Writer software

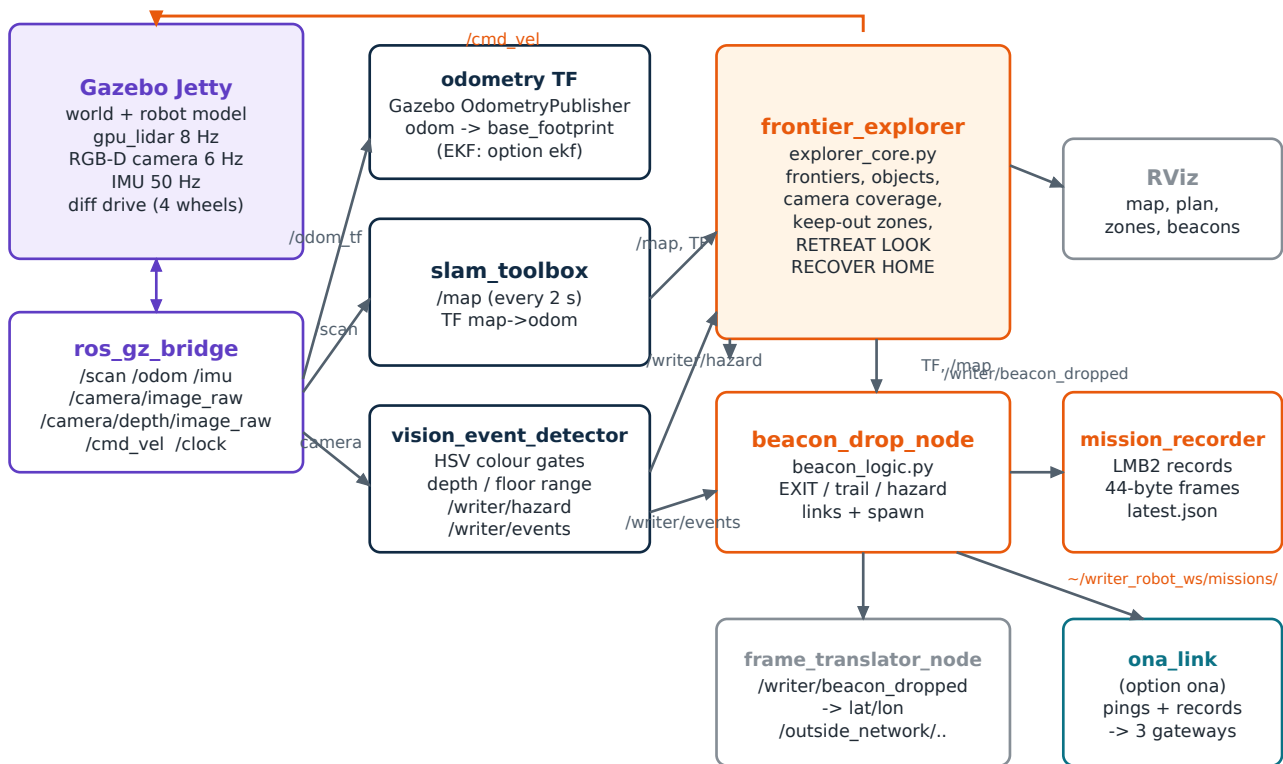


Figure 4. Writer processes and the main topics between them.

4.1 Topics

topic	type	from → to	content
/scan	LaserScan	bridge → SLAM, explorer	180 beams, 8 Hz
/odom, /imu	Odometry, Imu	bridge → vision, beacons, recorder, ona_link	true motion, gyro
/tf odom→base_footprint	TF	bridge (/odom_tf) → everyone	the robot's motion, 30 Hz
/camera/image_raw, /camera/depth/image_raw	Image	bridge → vision	RGB and depth
/map	OccupancyGrid	SLAM → explorer, beacons	5 cm cells, every 2 s
/writer/hazard	String JSON	vision → explorer	every sighting, ≤ 5 Hz per type
/writer/events	String JSON	vision → beacons, explorer	once per hazard, within 4 m
/writer/beacon_dropped	String JSON	beacons → recorder, translator	id, type, drop x/y, link_id, hazard x/y
/writer/mission_record	String JSON	recorder → ona_link	the LMB2 record + frame hex
/cmd_vel	Twist	explorer → bridge	10 Hz
/writer/plan	Path	explorer → RViz	current path
/writer/explorer_markers	MarkerArray	explorer → RViz	zones, goal, state text
/writer/explorer_status	String	explorer → ona_link	e.g. RETREAT: 6 waypoints left
/writer/beacon_markers	MarkerArray	beacons → RViz	beacon posts with ids, links to the way out
/outside_network/beacon_gps	String JSON	translator	beacon as lat/lon

topic	type	from → to	content
/trapdoor_cmd	Float64	beacons → Gazebo	servo angle (−1.2 open, 0 closed)
/ona_link/status	String	ona_link	pings sent, records, briefings, simulation speed

4.2 What run_explorer.sh starts

step	process	log
0	kills anything left from a previous run	–
1	ros2 launch writer_robot gazebo_sim.launch.py world:=... (Gazebo, robot, bridges incl. the odometry TF, robot_state_publisher); waits for /scan and the odometry TF	/tmp/sim.log
2	slam_toolbox, configure + activate; waits for map→odom	/tmp/slam.log
3	vision_event_detector, beacon_drop_node, mission_recorder, frame_translator_node	/tmp/vision.log, beacon.log, mission.log, frame.log
3b	option ona: the ONA (python -m ona, from the copy in the package, posting to COMMAND_POST, default http://10.0.2.2:3000) and ona_link	/tmp/ona.log, /tmp/ona_link.log
4	frontier_explorer (drives the robot)	/tmp/explorer.log
5	RViz with config/writer.rviz	/tmp/rviz.log

Options: retreat / small (world), headless (no Gazebo window), ona, slow (0.5 m/s), gpu (VirtualBox 3D acceleration), ekf. Ctrl-C in that terminal stops every process (a trap calls the same clean-up as stop_demo.sh).

5. The explorer brain (explorer_core.py)

explorer_core.py contains no ROS code. The ROS node frontier_explorer.py feeds it the map, the scan, the pose from TF and the hazards, and sends its velocity command to /cmd_vel. The 2D simulator feeds the same class, which is why the Gazebo robot and the tests run identical logic.

5.1 Planning grid

- The SLAM map is resampled to a 0.10 m planning grid. Unknown cells are **not** passable for the Writer.
- Obstacles are inflated with an exact Euclidean distance transform: the robot centre stays ≥ 0.36 m (robot_radius) from every occupied cell, so 0.9 m gaps stay passable. A soft cost keeps paths about 0.65 m from walls when there is room.
- Fresh LiDAR hits within 5 m are injected into the grid before planning, so an obstacle not yet in the 2 s map still blocks the path. Points inside the robot footprint (self-hits) are ignored.
- Paths are planned with Dijkstra from the robot cell to all cells at once, which also prices every candidate target in one pass.

5.2 Targets

target kind	what it is	why
LiDAR frontier	cluster (≥ 5 cells) of free cells next to unknown space, not hugging a wall	classic exploration: the map grows
object	isolated obstacle up to 1.2 m across that the camera has not looked at	the LiDAR cannot tell rubble from a hazard block: the camera must look

target kind	what it is	why
camera coverage	floor patch (≥ 3 blocks of 0.3×0.3 m) not yet inside the camera cone (0.9 rad, 3.2 m)	hazards are only recognised by the camera, so every room must be looked at

The cheapest target wins: path length, plus 3 m for camera-only targets (frontiers first) and 1 m for objects. A goal that fails twice is blacklisted (0.7 m radius). When no target is left, the map is complete.

5.3 Path following

Pure pursuit with a 0.70 m look-ahead point found by projecting the robot onto the path and intersecting a circle with the next segments. At sharp corners it drives to the corner first ("corner capture") instead of cutting it. Before any turn on the spot, a turn guard checks the 0.30 m circle it sweeps and creeps away from anything inside it. The command law is continuous:

- $\text{speed} = 0.70 \text{ m/s} \times \cos^2(\text{heading error})$, capped by $0.12 \text{ m/s} + 0.5 \times \text{distance to the goal}$ (gentle arrival; the Executor drives through intermediate beacons without it), $0.12 \text{ m/s} + 0.8 \times \text{distance to the next sharp corner}$ (slow into corners, only when a corner is ahead), and $0.10 \text{ m/s} + 1.0 \times (\text{free space ahead} - 0.30 \text{ m})$ (the LiDAR stop line is never reached fast);
- speed-up limited to 1.0 m/s^2 , braking immediate; turn rate $\leq 1.6 \text{ rad/s}$, changing by at most 3 rad/s^2 . The ramps use the real time between two control ticks, so a busy computer that runs the node less often does not change them;
- turning on the spot starts above 1.0 rad of heading error and stops below 0.5 rad (hysteresis: no chattering);
- when the current goal becomes useless while driving (frontier already explored, area already seen), the next goal is planned on the move instead of stopping first.

In the 2D simulator the big map is explored in about 7.3 simulated minutes, with no collision, measured with slam_toolbox's limit of one scan every 0.5 s. The option `slow` (0.5 m/s, slower turns) is there for a computer on which the faster robot bumps into things.

5.4 Hazards, keep-out zones and the retreat

- Every `/writer/hazard` sighting within 1.0 m of a known hazard of the same type refines it; otherwise it is a new hazard with a **1.2 m keep-out disc** that no path and no goal may enter. A 1.2 m disc closes a corridor, so the robot takes the long way round.
- If the robot is already inside a disc, it may only move away from the hazard.
- **Retreat trigger:** the new hazard is closer than 1.8 m (disc + 0.4 m + 0.2 m), or the current path passes within 1.6 m of it.
- **Retreat:** reverse 0.3 m, turn round, and drive back along the recorded trail (points every 0.15 m) for at least 1.5 m and until 2.5 m from every hazard (at most 12 m). Then plan again: the keep-out zone forces another route.
- A hazard that appears during a retreat only matters if the trail leads into it; then the robot replans.

5.5 States

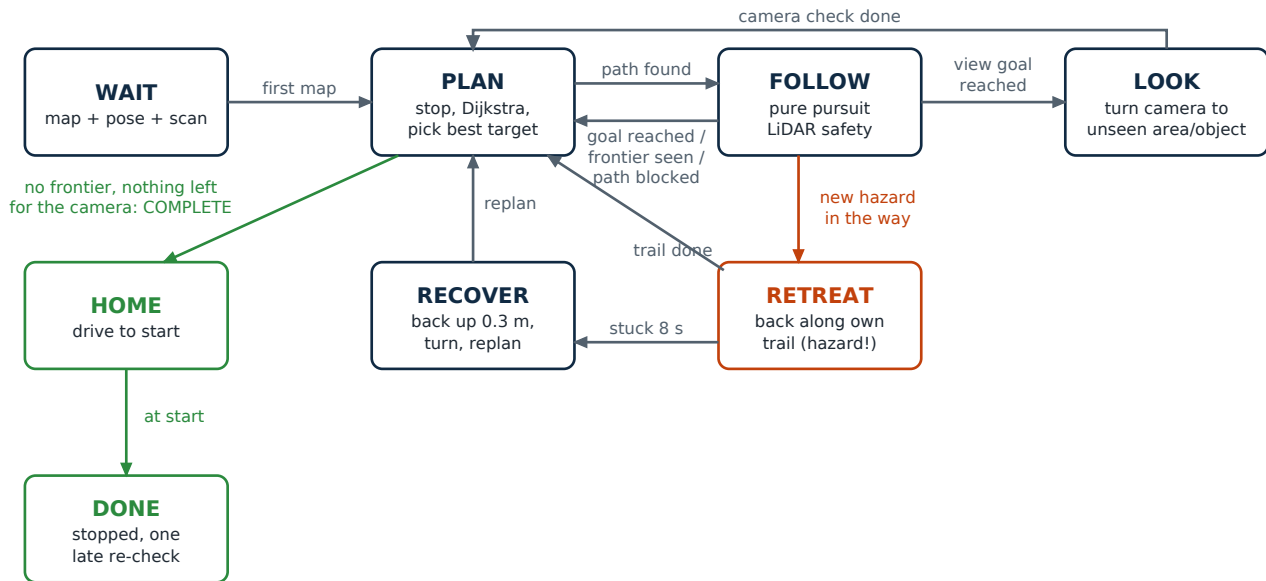


Figure 5. Writer state machine.

state	what happens
WAIT	until the first map, a pose and a scan exist (plus <code>start_delay_s</code> , 3 s)
(first turn)	one continuous full turn on the spot at 1.0 rad/s before the first plan. <code>slam_toolbox</code> adds a scan only after 0.3 m or 0.3 rad of motion, at most one per 0.5 s, and marks a cell only when several rays crossed it, so a robot standing still has a map of barely 1 m around itself; the turn adds ~12 scans. If the map is still tiny, the robot drives ~0.6 m toward the most open direction the LiDAR sees (NUDGE, up to 3 times). A warning is logged if the TF heading changed less than 170° during a full turn.
PLAN	stop, wait up to 2.5 s for a newer map after reaching a goal, pick the cheapest target, plan
FOLLOW	pure pursuit; replans when the goal is reached, the frontier has been seen or the path is blocked
LOOK	turn at up to 1.6 rad/s until the camera faces the unseen patch (within 0.25 rad) and go straight on to the next heading, at most 3 per stop: the turn itself sweeps the camera over the area (coverage counts a 0.9 rad cone, the real view is 1.09 rad). Only before turning back does it hold 0.25 s. An object: face it within 0.12 rad and hold 0.35 s. About 2 s per stop.
RETREAT	back along its own trail, away from a hazard
RECOVER	no progress (0.15 m along the path) for 8 s, or told to move for 2 s without moving, or the LiDAR stop 3 times: back up 0.3 m at 0.2 m/s, turn ~80° to the open side, replan
HOME	EXPLORATION COMPLETE: drive back to the start. If nothing is reachable while the robot has driven less than 1 m, it turns on the spot again instead (at most twice). A <code>map check</code> : line in the log gives map size, free and reachable area and frontier cells.
DONE	stopped at the start; 12 s later one last look at the newest map in case something appeared

Every simulated minute the log gets a line such as `time so far 4.0 min: driving 170 s, camera looks 38 s, planning 14 s, recovering 6 s; driven 96 m (Writer and Executor)`, so a slow run shows at once where the time goes: `grep "time so far" /tmp/explorer.log`.

5.6 Safety net (always on)

- Forward motion stops if the LiDAR sees anything closer than 0.30 m in a $\pm 35^\circ$ front sector.
- Reversing needs 0.45 m free behind.
- A /scan older than 1.5 s stops the robot.

5.7 Parameters (frontier_explorer)

Set with `-p name:=value` (see the Guide) or in `run_explorer.sh`. Defaults are read from the code.

parameter	default	meaning
plan_res	0.1	planning grid resolution (m)
robot_radius	0.36	hard clearance for the robot centre (m)
footprint_radius	0.3	circle swept when turning on the spot (m)
soft_radius	0.65	paths prefer to stay this far from walls (m)
soft_weight	3	weight of the soft wall cost
keepout_radius	1.2	no-go disc around each hazard (m)
hazard_merge_dist	1	sightings closer than this = same hazard
retreat_path_margin	0.4	retreat if the path passes within keepout+this
retreat_clear	2.5	retreat until this far from every hazard (m)
retreat_min	1.5	...and at least this far back along the trail
retreat_max	12	longest retreat (m)
backup_dist	0.3	straight reverse before turning round
lookahead	0.7	pure-pursuit look-ahead (m)
v_max	0.7	top speed (m/s)
v_min	0.1	lowest driving speed (m/s)
w_max	1.6	top turn rate (rad/s)
rotate_in_place	1	heading error (rad) above which we turn on the spot
rotate_exit	0.5	...and below which we drive again (hysteresis: no chattering)
accel	1	m/s ² : smooth speed-up (braking is immediate)
corner_slow	0.8	m/s per m before a sharp corner (0.12 m/s at the corner)
goal_tolerance	0.3	goal reached within (m)
frontier_min_cells	5	min frontier cluster size (planning cells)
frontier_wall_cells	3	unknown cells this close to a wall are not frontiers
goal_search_cells	4	goal may be this many cells from its frontier
min_goal_dist	0.45	frontiers closer than this are considered reached
blacklist_radius	0.7	area blacklisted around a failed goal (m)
stuck_time	8	s without progress along the path -> recover
stuck_progress	0.15	progress needed in stuck_time (m)
blocked_time	2	s commanded to move while the robot does not move -> recover at once
recover_speed	0.2	m/s backing up in a recovery
emergency_dist	0.3	LiDAR range in the front sector that stops forward motion
emergency_sector_deg	35	half-width of the stop sector (deg)
rear_clear_dist	0.45	free space needed to reverse (m)
lidar_x	0.1	LiDAR position ahead of base_footprint
self_box	-0.34, 0.13, -0.22, 0.22	robot footprint in the LiDAR frame (x0, x1, y0, y1)
scan_inject_range	5	add fresh LiDAR hits to the planning grid
history_step	0.15	trail recorded every (m)

parameter	default	meaning
map_wait	2	after reaching a frontier, wait for a newer map (s)
home_on_finish	true	drive back to the start at the end
coverage	true	camera coverage on (false = LiDAR exploration only)
camera_x	0.213	camera ahead of base_footprint (m)
camera_fov	0.9	horizontal FOV used for coverage (rad, < real 1.089)
camera_range	3.2	hazards are recognised reliably within this (m)
cov_block	3	coverage bookkeeping in blocks of 3x3 planning cells
cov_min_blocks	3	ignore unseen patches smaller than this many blocks
cov_penalty	3	m: LiDAR frontiers are preferred over camera-only targets
cov_view_dist	2.2	viewpoint must be within this of the unseen patch (m)
object_max_size	1.2	isolated obstacles up to this size may be hazards...
object_penalty	1	...and are looked at (camera centred on them) first
look_dwell	0.35	s facing a suspicious OBJECT so the camera gets frames (6 Hz: 2 frames)
sweep_dwell	0.25	s held at an AREA look heading only before turning BACK: turning on,
look_w_max	1.6	rad/s when turning the camera toward an area
look_tol	0.25	rad: an area look needs no precise heading
look_max	3	headings per camera stop
spin_rate	1	rad/s of the full turn at the start (slam_toolbox takes a scan per 0.5 s)
time_report	60	s between "time so far" lines in the log (0 = never)
verbose	false	log every planning decision
unknown_is_free	false	plan through unknown space (Executor only)
unknown_cost	1.5	extra cost of unknown cells (Executor)
initial_spin	true	
map_pad	0.6	unknown margin around the SLAM map (m): frontiers at its edge count

6. Vision: seeing the hazard blocks

`vision_event_detector.py` finds the coloured hazard blocks in the camera image, measures how far they are, places them on the map and tells the explorer and the beacon node. The HSV masks stand in for a trained detector (YOLO / MobileNet on a Jetson) later; the rest of the chain would not change.

6.1 Pipeline, per colour frame

- 1 BGR → HSV, then one mask per hazard type (table below), cleaned with a 5 × 5 morphological opening.
- 2 Largest contour of each mask; ignored below `min_area_px` (800 px ≈ a block at 3.7 m).
- 3 Bearing of the blob centre from the pinhole model: $fx = (w/2) / \tan(HFOV/2)$, bearing = $\text{atan2}(w/2 - cx, fx)$.
- 4 **Depth**: median of the depth image over the blob (≥ 20 valid pixels; 16-bit images are millimetres).
- 5 **Fallback when there is no usable depth**: the block stands on the floor, so its lowest image row gives the range: $\text{depth} = \text{camera height} \times fx / (\text{row} - cy)$. If the bottom is cut by the image edge, the

block's known width is used: $\text{depth} = 0.30 \text{ m} \times \text{fx} / \text{pixel width}$. A slow VM often delivers colour frames without a matching depth frame, so without this fallback a block could go unreported.

- 6 The hazard is projected into the map from the robot's TF pose at the image time stamp.
- 7 `/writer/hazard`: every sighting ($\leq 5 \text{ Hz}$ per type) for the explorer. `/writer/events`: once per hazard (no second event within 1.5 m, 3 s cool-down), when it is closer than `drop_range_m` (4 m), for the beacon node.

type	block colour	HSV gate (OpenCV, H 0-179)	LMB2 kind
radiation	magenta / purple	H 140-170, S ≥ 90 , V ≥ 80	RADIATION
fire	red	H 0-10 or 170-179, S ≥ 120 , V ≥ 90	THERMAL
gas	yellow	H 18-35, S ≥ 150 , V ≥ 90	GAS

Walls, rubble, the green beacon bodies and their blue LEDs fall outside every gate.

6.2 Health log

Every 10 s the node writes one line to `/tmp/vision.log`, for example:

```
[health] last 10 s: 58 colour frames, 57 depth frames, hazard frames: fire 12, radiation 3
[health] no depth images on /camera/depth/image_raw - using the floor-contact range estimate instead
[health] NO camera images on /camera/image_raw - is the bridge running?
```

Zero colour frames means the camera bridge is not running; zero depth frames means ranges come from the fallback; a colour that never appears in "hazard frames" while the block is in view means its HSV gate or `min_area_px` needs tuning (Guide, troubleshooting).

6.3 Parameters (`vision_event_detector`)

parameter	default	meaning
<code>min_area_px</code>	800	smallest blob, in pixels ($\approx 3.7 \text{ m}$ for a $0.3 \times 0.6 \text{ m}$ block)
<code>drop_range_m</code>	4.0	a beacon is recorded only for hazards closer than this
<code>cooldown_s</code>	3.0	minimum time between two events of the same type
<code>dedup_radius_m</code>	1.5	one event per hazard within this radius
<code>hazard_rate_hz</code>	5.0	sightings per second per type on <code>/writer/hazard</code>
<code>hfov_rad</code>	1.089	camera horizontal field of view
<code>camera_offset_m</code>	0.213	camera ahead of base_footprint
<code>camera_height_m</code>	0.135	optical centre above the floor (fallback)
<code>hazard_width_m</code>	0.30	block width (fallback)

7. Beacons: dropping, linking, recording

7.1 When a beacon is dropped (`beacon_drop_node` + `beacon_logic.py`)

beacon	rule
EXIT	one, at the start (the entrance and the way out), the root of the tree
trail	after a heading change $\geq 35^\circ$ once $\geq 0.8 \text{ m}$ from the last beacon, or every 2.5 m of straight driving
hazard	on each <code>/writer/events</code> message, where the robot stands (the record carries the hazard's own position)

beacon	rule
no duplicate	no trail beacon within 1.2 m of an existing one that is in line of sight (checked on <code>/map</code>), for example when driving back along the trail

The beacon falls from the magazine 0.12 m behind the robot centre. In Gazebo the node spawns a static, visual-only `beacon_<id>` model on the floor (no collision box, so it can never jam the robot) and pulses the trapdoor servo. The magazine holds 120 beacons.

7.2 How beacons are linked: the tree

Each beacon's **next** is the **last beacon dropped or driven past** (within 0.5 m). It is never a beacon left far behind, and never one behind a wall (line of sight on the SLAM map). The result is a tree rooted at the EXIT whose links are short drivable legs (≤ 3.7 m).



Figure 6. How the links form a tree. After the retreat, the new branch hangs from #2.

7.3 The `/writer/beacon_dropped` message

```
{ "beacon_id": 5, "event_type": "waypoint", "value": 0.0,
  "x": 3.417, "y": -2.94, "theta": 1.57,           <- where the beacon lies (map frame)
  "link_id": 2,                                   <- its way out = LMB2 next
  "event_x": ..., "event_y": ..., "range_m": ..., <- hazard beacons: the hazard itself
  "timestamp": 1790000088.2, "ttl": ..., "magazine_remaining": 114 }
```

7.4 From a beacon to a signed record (`mission_recorder + mission_log.py`)

record field	how it is filled
beacon_id	the beacon id (0 = EXIT)
kind	exit → EXIT, waypoint → WAYPOINT, radiation → RADIATION, fire → THERMAL, gas → GAS, victim → VICTIM
gps.lat/lon	map (x, y) → east/north (rotated by <code>map_yaw_deg</code>) → lat/lon around the anchor (default 36.8065 N, 10.1815 E, Tunis). Hazard beacons use the hazard position.
gps.err_m	0.3 m + 2 cm per metre driven so far (+ 25 % of the camera range for hazards), ≤ 25.4 m
severity	100 × blob size value for hazards, 0 otherwise
confidence	0.9 for camera detections, 1.0 for exit and trail
next	id of <code>link_id</code> , distance and compass bearing from this beacon to it
ts, TTL, half-life	record time; lifetime and half-life from the kind table (section 8.2)

The record is encoded as the 44-byte LMB2 frame with the mission key (demo = the demo key) and network id 0x2A, then appended to `~/writer_robot_ws/missions/latest.json` and a dated copy `mission_<date>.json`. The file is rewritten atomically after every beacon, so a crash never leaves a half-written mission.

```
{
  "format": "living-map-mission/1",
  "anchor": {
    "lat": 36.8065,
    "lon": 10.1815,
    "map_yaw_deg": 0.0,
    "net_id": 42,
    "key": "demo",
    "started": 1790000000,
    "updated": 1790000757,
    "world": "contaminated_zone.world",
    "home": [0.0, 0.0],
    "beacons": [
      {
        "id": 5,
        "event_type": "waypoint",
        "drop": [3.417, -2.94],
        "event": [3.417, -2.94],
        "record": {
          "beacon_id": 5,
          "kind": "WAYPOINT",
          "gps": {
            "lat": 36.8064736,
            "lon": 10.1815383,
            "err_m": 0.6,
            "src": "slam",
            "severity": 0,
            "confidence": 1.0,
            "ts": 1790000088,
            "next": {
              "id": 2,
              "dist_m": 3.5,
              "bearing_deg": 265.0
            }
          },
          "frame": "212a05000f00050001000003e038f015579411060600ffbc02002300d83bb16ac021e01037f852f084ff7484",
          ...
        }
      }
    ]
  }
}
```

NOTE The Executor uses only the frame strings (it verifies and decodes them itself). drop and event are there for `mission_world`, which places the beacon models in Gazebo, and for the test tools.

8. The LMB2 beacon network

LMB2 ("Living Map Beacon, protocol 2") is the radio protocol and the replication engine that run on every beacon, on the gateway and inside the simulator. The C core (`lib/BeaconProto`) is portable C99 with no heap; the firmware, the gateway and `bpsim` compile the very same files, and a byte-exact Python mirror is used by the bridge and the robots.

8.1 The 44-byte frame

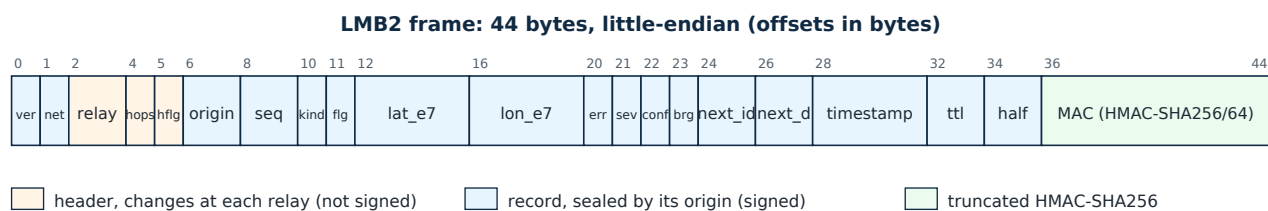


Figure 7. Frame layout.

off	size	field	meaning	signed
0	1	ver_type	0x21 = protocol 2, record (0x22 digest, 0x23 robot, 0x24 mission: section 10.6)	yes
1	1	net_id	team / mission network, default 0x2A	yes
2	2	relay_id	node that transmitted this copy	no
4	1	hops limit	hops so far (high nibble), hop limit (15 = flood everywhere)	no
5	1	hdr_flags	ORIGIN_SILENT / ORIGIN_ALIVE liveness hints	no
6	2	origin_id	the beacon that owns the record	yes
8	2	seq	version counter (serial-number arithmetic, survives wrap)	yes
10	1	kind	WAYPOINT 0, VICTIM 1, GAS 2, RADIATION 3, THERMAL 4, OBSTRUCTION 5, STRUCTURAL 6, EXIT 7; mine: PHOSPHATE 8, GOLD 9, GEMSTONE 10, SEARCHED 11	yes
11	1	flags	HAS_NEXT, HAS_BEARING, POS_GNSS, RETRACTED, STALE	yes
12	8	lat_e7, lon_e7	degrees × 10 ⁷ (1.1 cm)	yes
20	1	err_dm	position error, 0.1 m steps (255 = unknown)	yes
21	2	severity, confidence	0-255 each; confidence at the time stamp	yes

off	size	field	meaning	signed
23	1	next_bearing	compass bearing to the next hop, 1.4° steps	yes
24	4	next_id, next_dist_dm	next beacon on the way out and its distance (0.1 m)	yes
28	4	timestamp	unix seconds when the record was made	yes
32	4	ttn_10s, half_life_10s	hard lifetime and confidence half-life, 10 s steps	yes
36	8	mac	HMAC-SHA256(key, "LMB2" bytes 0-1 bytes 6-35), first 8 bytes	-

8.2 Kinds and adaptive aging

A record states how sure its origin was when it was made. Every node computes the current value locally: $\text{eff_conf} = \text{confidence} \times 2^{(-\text{age} / \text{half-life})}$. A record is forgotten when its age passes the TTL or eff_conf drops below 0.05. The same value drives cache eviction and background priority: $\text{weight}(\text{kind}) \times (1 + \text{severity}/64) \times \text{eff_conf}$, $\times 1.5$ for a beacon reported lost.

kind	half-life	TTL	weight	note
WAYPOINT	12 h	24 h	1.0	trail breadcrumb
VICTIM	1 h	2 h	3.0	highest priority
GAS	10 min	1 h	2.5	disperses fast
RADIATION	6 h	12 h	2.5	
THERMAL (fire)	20 min	1 h	2.0	
OBSTRUCTION	48 h	7 d	1.5	rubble does not move
STRUCTURAL	24 h	3 d	2.0	collapse risk
EXIT	48 h	7 d	1.5	safe exit
PHOSPHATE (mine)	72 h	7 d	0.8	severity = % $\text{P}_2\text{O}_5 \times 4$
GOLD, GEMSTONE (mine)	72 h	7 d	0.8	a find to sample
SEARCHED (mine)	2 h	12 h	1.2	area searched; severity = % seen

Updates keep the origin and bump seq (a victim getting worse, a hazard retracted). When a beacon's own event expires it does not go silent: it re-issues its record as a **STALE route marker** with the same position and next, so the route to the exit is never broken.

8.3 Security

- **Authenticity and integrity:** 64-bit truncated HMAC-SHA256 under a 128-bit mission key, domain tag "LMB2". The MAC also covers the version and the network id. A blind forgery succeeds with probability 2^{-64} .
- **Replays:** a replayed (origin, seq) is a duplicate; an older seq is dropped; time stamps more than 5 min in the future and expired records are refused. A beacon persists its seq in flash and bumps it at every provisioning.
- **Unsigned header:** relay_id, hops and the liveness hints change at every hop. Abusing them can at worst skip one relay (repaired by acks and anti-entropy) or raise a false "lost?" hint for beacons nobody hears strongly.
- **Keys:** one key per mission, loaded over USB (KEY <32 hex>) and stored in NVS. KEY demo is for demos only. A captured beacon reveals the shared key: rotate keys per mission, enable ESP32 flash encryption.
- No confidentiality by design (rescue data is not secret). Jamming is out of scope.

8.4 Radio

parameter	value
frequency	868.1 MHz (EU 868.0–868.6 MHz sub-band: 25 mW e.r.p., 1 % duty cycle). Tunisia: confirm the band with the regulator; BP_LORA_FREQ_HZ changes it.
modulation	LoRa SF9, 125 kHz, CR 4/5, explicit header, CRC; 288 ms per 44-byte frame; –129 dBm sensitivity
power, sync word	14 dBm; private sync word 0x12 (LoRaWAN traffic is filtered by the radio)
duty budget	1 % = 36 s per hour = 125 frames per hour at SF9, enforced by a token bucket
other SF	SF7: 93 ms per frame, 31 s round; SF12: 2.14 s per frame, 12 min round

8.5 The gossip engine

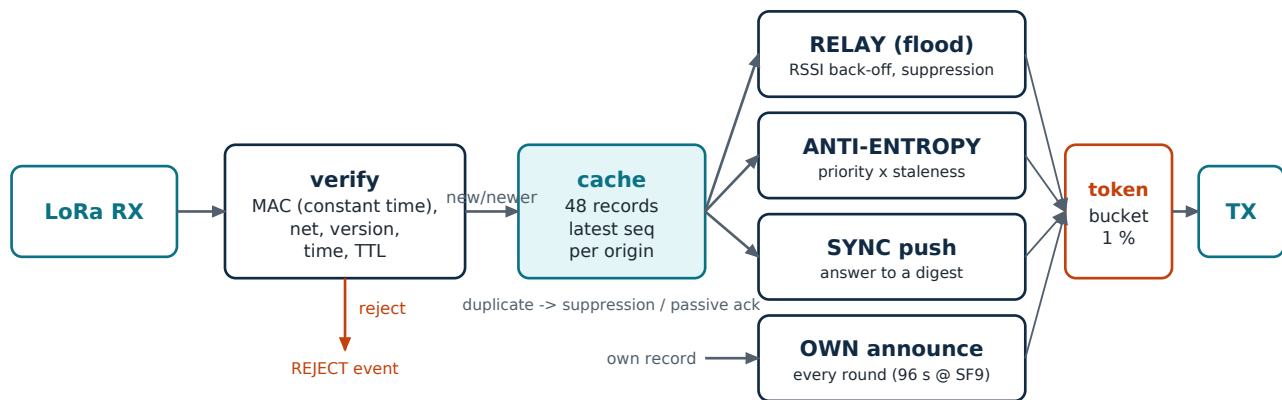


Figure 8. Inside one beacon (bp_gossip.c).

mechanism	what it does
own announce	every round (96 s at SF9, ± 20 % jitter) the beacon sends its own record. Round = 2 frames / $(0.6 \times \text{duty})$: announce + repair use 60 % of the budget, 40 % stays free for floods.
reactive relay	a never-seen (origin , seq) is re-sent once after an RSSI-weighted back-off (far nodes first). It is cancelled if a copy is heard from a node farther along the route tree (directional suppression). After relaying, the node expects to overhear the next node repeat it (passive ack) and retries at most twice.
anti-entropy	each round, re-send the cached record with the best priority $\times$ (time since we last sent it). This keeps a destroyed beacon's record alive until its TTL.
pull digests	the gateway broadcasts a signed list of every (origin , seq) it holds (60 per page); neighbours push what is missing (≤ 2 records). A gateway switched on late catches up in ≤ 37 s (14 beacons).
liveness	a beacon is judged only through a strong link ($\text{RSSI} \geq \text{sensitivity} + 12$ dB and heard in ≥ 75 % of rounds). 5 silent rounds $\rightarrow$ ORIGIN_SILENT; the dashboard shows "LOST?" 7–10 min after a beacon is destroyed, with 0 false alarms in 40+ runs.
duty control	token bucket, provably ≤ 1 % in any rolling hour; priority classes RELAY > SYNC > new OWN > periodic OWN > ANTI-ENTROPY, with a reserve so background traffic never starves a flood.
route seeding	the Writer primes each new beacon over USB with the records it already has (SEED), so a fresh beacon knows the route tree at once (worst-case latency 1728 s $\rightarrow$ 106 s).

8.6 What happens when things break

failure	network behaviour
a beacon is crushed	its record is already in every neighbour and keeps circulating; neighbours report it lost; the dashboard keeps the pin and the route through it
one frame lost	passive-ack retry, then anti-entropy
gateway reboots or arrives late	pull digests rebuild its copy
an event ages out	STALE route marker keeps position and next hop
forgery, tampering, replay	rejected (0 accepted in every simulated run)
network cut in two	old records survive on both sides; new ones cannot cross. Fix: drop a replacement beacon (new id), or let the Executor carry the data across

8.7 Provisioning a real beacon

`beacon_record_node.py` (`beacon_net/ros/`) listens to `/writer/beacon_dropped`, builds the record exactly as in section 7.4, and talks to the beacon in the dispenser over USB serial (115200 baud): `TIME <unix>`, `PROV {record json}`, then `SEED {older record}` for the route so far. The beacon announces within 0.8 s and stores its record and seq in flash. Ids are never reused.

serial command	effect
<code>KEY <32 hex> / KEY demo</code>	mission key (persisted)
<code>NET <n> · ID <n> · NAME <s></code>	network id, node id, gateway label
<code>TIME <unix></code>	set the clock (beacons have no RTC)
<code>PROV {json}</code>	install this beacon's record
<code>SEED {json}</code>	prime the cache with a known record
<code>RETRACT</code>	withdraw the event: RETRACTED flag, new seq
<code>STATUS · DUMP · VERBOSE 0/1 · REBOOT · FACTORY · HELP</code>	inspection and maintenance

8.8 Hardware

ESP32 + SX1276 (TTGO LoRa32 pinout by default, any RFM95 with 6 `#defines`), 868 MHz whip antenna, one 18650 cell. About 40 mA on average (ESP32 at 80 MHz + continuous receive + < 1 % transmit): about 3 days per 3000 mAh. Light sleep with radio wake-up would give more than a week.

8.9 Verification of the network

- `bpsim`: every node runs the real `bp_gossip.c` on a simulated channel (path loss, debris, corridor turns, fading, collisions, capture, half-duplex). Scenario: 14 beacons along a winding corridor, beacon 6 destroyed at minute 25, a victim update at minute 30, an attacker every 90 s. 13 automatic checks.
- Stress matrix (`make matrix`): **42 / 42 PASS** (14–38 beacons, heavy rubble, SF7–SF12, late gateway, ...).
- Cross-implementation tests: C = Python byte for byte on 400 random records and 300 JSON inputs, HMAC vectors, airtime table, and the real firmware compiled for the PC.

9. Gateways and the Command Post

9.1 The gateway board

Same board as a beacon, flashed with `firmware/gateway_node`, on USB to the ONA computer (three of them around the building) or, without the ONA, to the Command Post laptop. It is a passive member of the network: it verifies every frame, keeps a replica, pulls anything it misses, and prints one JSON object per line. For the ONA it also prints every 44-byte frame it hears, raw, so the ONA can check it with its own key:

```
{
  "ev": "rx",
  "gw": "gw1",
  "frame": "212a0700...(88 hex)",
  "rssi": -97,
  "snr": 4.5,
  "ts": 1790345900
} <- raw frame
{
  "beacon_id": 7,
  "seq": 2,
  "kind": "VICTIM",
  "gps": {
    "lat": 34.4303,
    "lon": 8.7833,
    "err_m": 1.9,
    "src": "slam"
  },
  "severity": 40,
  "confidence": 0.96,
  "next": {
    "id": 6,
    "dist_m": 8.6,
    "bearing_deg": 180.0
  },
  "ts": 1790345900,
  "ttl_s": 7200,
  "half_life_s": 3600,
  "retracted": false,
  "stale": false,
  "ev": "update",
  "rx_node": 900,
  "age_s": 1,
  "eff_conf": 0.960,
  "hops": 0,
  "relay": 7,
  "rssi": -115,
  "snr": 2.0,
  "silent": false
}
{
  "ev": "reject",
  "reason": "bad_mac",
  "rx_node": 900,
  "ts": 1790345960
}
{
  "ev": "hb",
  "gw": "gw1",
  "known": 14,
  "rx_ok": 212,
  "rx_bad": 3,
  "digests": 9,
  "duty": 0.004
}
```

serial command	effect
KEY <32 hex> / KEY demo · NET <n> · NAME gw1 · TIME <unix>	mission key, network id, gateway name, clock (the ONA sends them itself)
TX <88 hex>	transmit one 44-byte frame for the ONA (a briefing), charged to the 1 % duty budget: OK TX type 0x24 or ERR ...
RAW 0 / RAW 1	raw frame lines off / on (default on)
STATUS · DUMP · VERBOSE 0/1 · HELP	as for a beacon (section 8.7)

9.2 One gateway without the ONA: gateway_bridge.py

Standard-library Python (plus `pyserial` for a real board). It reads one gateway's JSON lines and posts each record to the dashboard, keeping the original contract fields (`event_type = kind`, `value = severity`, `ttl`) and adding the full record (`v: 2`), health and a time scale for accelerated simulations. It is the quickest way to show the beacon network on its own; with three gateways, the ONA (section 10) replaces it.

source	use
--serial PORT	a real gateway (COM3 on Windows, <code>/dev/ttyUSB0</code> in Linux); sends KEY and TIME itself
--stdin	anything piped in, e.g. <code>./bpsim --speed 20 ... --stdin</code>
--sim "ARGS"	starts <code>bpsim</code> itself with these options
--exec CMD	starts any command and reads its output

9.3 The Command Post dashboard

Node.js + Express + Socket.io server, Leaflet map in the browser (`command_post/`). With no internet at the venue it switches to an offline grid by itself.

- **Live map:** one pin per beacon, coloured by kind; opacity follows each record's half-life, extrapolated live. Dashed arrows are the `next` hops; opening a pin highlights its whole route to the exit and its error circle.
- **Votes:** every record from the ONA carries its vote (`✓3/3`, `✓2/3`, ...`1/3`). A version only one gateway reported is dotted and faded, not counted in the tiles, and never replaces a confirmed version.
- **Gateways and robots:** the three gateways as antennas; around them, each one's measured distance to a robot as a circle (the "spheres"); the robot where they cross, inside its 95 % error ellipse; a small dashed circle where the robot thinks it is. **Spheres** toggles the circles.

- **Panels:** Outside network area (uplink LTE / SATELLITE / NO UPLINK, each gateway's agreement, records confirmed / waiting / conflicts, queue, frame check, briefing); Executor and Writer (measured by N gateways $\pm$ error, HDOP, own SLAM agrees or DRIFT, phase, battery, mission progress).
- **Lost beacons** get a red dashed ring and "LOST?"; stale route markers are hollow; retracted hazards grey.
- **Feed** (beacon events and ONA alerts with toasts), **gateway chips** in the top bar (records held, HMAC rejects, IGNORED), stats tiles.
- **Mission planner:** select pins, reorder, *Dispatch mission*, with "back to the exit afterwards"; *Executor path to here* turns a beacon's way-out chain, reversed, into waypoints; *Call the Executor back* sends a signed ABORT.

method	path	effect
POST	/api/beacon	store (latest record per beacon, 200 beacons; an unconfirmed version is kept aside) and push <code>beacon:new</code>
POST	/api/executor-status	push <code>executor:update</code> (Executor position and heading)
POST	/api/network-health	push <code>network:health</code> (one gateway)
POST	/api/robot-status	from the ONA: a robot's measured position, error ellipse, ranges per gateway, own pose, consistency, phase, progress → <code>robot:update</code>
POST	/api/ona-status	from the ONA: uplink, queue, gateways, votes, calibration check, briefing → <code>ona:status</code>
POST	/api/ona-event	from the ONA: alerts and decisions (section 10) → <code>ona:event</code>
GET	/api/beacons	last 200 beacons
POST	/api/mission-dispatch	store the latest mission (waypoints, <code>return_to_exit</code> , or abort)
GET	/api/mission-dispatch/latest	the last dispatched mission (the ONA polls it)
GET	/api/state	everything at once (used on load and reconnect)

9.4 Other ways into the dashboard

program	what it does
beacon_net/ros/beacon_record_node.py	Writer drops → LMB2 records (same links as the mission file) → directly to the dashboard (<code>-p dashboard:=...</code>) and/or a real beacon (<code>-p serial:=...</code>). A perfect radio network, no ONA.
command_post/ros_bridge.py	older path: <code>/writer/beacon_dropped</code> → dashboard in the original contract format, with a gw1 heartbeat
frame_translator_node	publishes each beacon as lat/lon on <code>/outside_network/beacon_gps</code> (ROS only)
<code>npm run demo / mock_feeder.js</code>	scripted mission with no robot: votes, both robots on their circles, a gateway that starts lying, a SLAM drift, the satellite link (<code>ONA=0</code> : one gateway)

10. The Outside Network Area

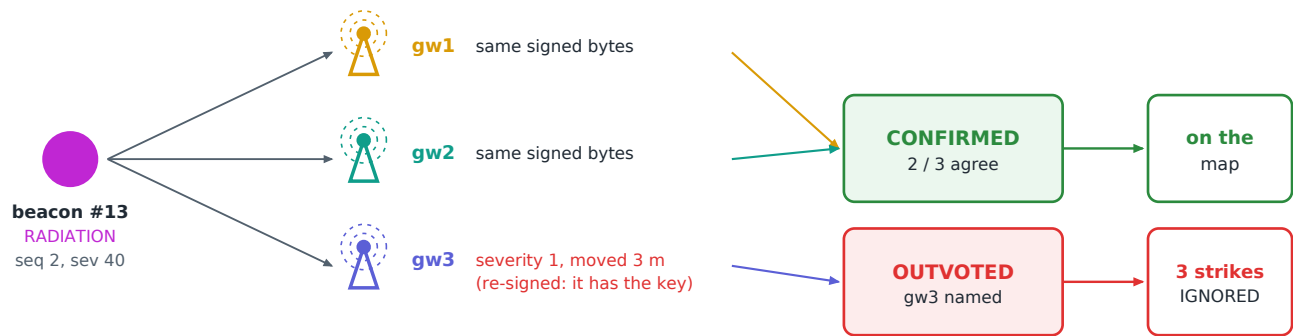
The brief asks for an Outside Network Area that connects the disconnected zone to the outside world: it must translate the robots' coordinates into GPS, forward the data to a distant Command Post over wireless or satellite links, and brief the Executor before entry. All communication between inside and outside passes through it. The ONA is built on the GALERIA team's code (frame translation, Kalman filter, persistent queue, LTE/satellite link, section 10.9) and adds three things no single gateway can give: **three gateways that vote**, **the robots located like GPS**, and **a physical cross-check** of what each robot says about its position.

file in ona/ona/	what it does
<code>lmb2.py</code>	the frames: RECORD, ROBOT, MISSION; signature checks (byte-identical to the beacons' codec)
<code>quorum.py</code>	the 2-of-3 vote, gateway scores, quarantine
<code>geodesy.py</code>	WGS84 / ECEF / ENU, Umeyama, the map → GPS calibrator (from <code>trans.py</code>)
<code>kalman.py</code>	PositionEKF (from <code>ekf.py</code>) and RangeEKF (one correction per gateway range)
<code>multilateration.py</code>	the three spheres: least squares, HDOP, RAIM
<code>tracker.py</code>	per robot: one ping's ranges, wall correction, filter, own pose vs measured, calibration check
<code>store.py</code> , <code>uplink.py</code>	SQLite queue and audit log, LTE first / satellite backup, HTTP to the Command Post (from <code>ona.py</code>)
<code>core.py</code> , <code>server.py</code>	the ONA logic; <code>python -m ona</code> : gateway inputs (USB serial, UDP, stdin), uplink thread, mission polling
<code>zonesim.py</code>	<code>python -m ona.zonesim</code> : a built-in building as three gateways hear it, with faults

10.1 Three gateways that vote

One gateway is a single point of failure and a single point of trust: if its USB link garbles a byte, if its firmware is old, or if somebody has tampered with it, the Command Post believes whatever it says. Signatures do not help against a gateway that holds the mission key: it can re-sign anything it likes. The ONA therefore never trusts one source.

- 1 Re-check.** Every frame a gateway passes on is checked again with the ONA's own copy of the key. A gateway whose frames keep failing is quarantined (3 strikes).
- 2 Vote.** The ONA compares the *signed content* (everything but the relay header) reported by the gateways for the same beacon and version. Two identical reports out of three: **CONFIRMED**, forwarded with its vote.
- 3 Name the liar.** A gateway that reported a different signed version than the majority gets a strike (**GATEWAY_DISAGREES**). Three strikes: **GATEWAY_QUARANTINED**, ignored from then on.
- 4 Wait, but do not hide.** A version only one gateway reported is held 4 s, then shown dotted as "1/3" (`--strict` hides it). It never replaces a confirmed version. Two versions and no majority for 10 s: **RECORD_CONFLICT**.



Only one gateway reports a version: it waits 4 s, then shows dotted "1/3" (never in strict mode), and never replaces a confirmed one.
Two versions and no majority for 10 s: RECORD_CONFLICT alert (a gateway is lying, or the key leaked).

Figure 9. The vote on one record, with a gateway that re-signs a changed copy.

NOTE A gateway may send a record twice, as its raw frame and as its decoded JSON line. Both are reduced to the same signed fields, so one gateway always counts once. What the vote does not stop: three gateways that all hear the same forged frame. The signature stops forged frames; the vote stops single points of failure.

10.2 Where is the robot? Three spheres

The robots have no GPS inside. Their own position comes from SLAM, in a private map frame, and SLAM can slip. The gateways stand outside at surveyed positions, and each measures its distance to the robot's radio by two-way time of flight. A distance is a sphere around the antenna. Three spheres, cut at the robot's antenna height, cross at one point. GPS needs four satellites because its receiver clock is a fourth unknown; two-way ranging has no clock offset, so three gateways give a position **and** one spare measurement.

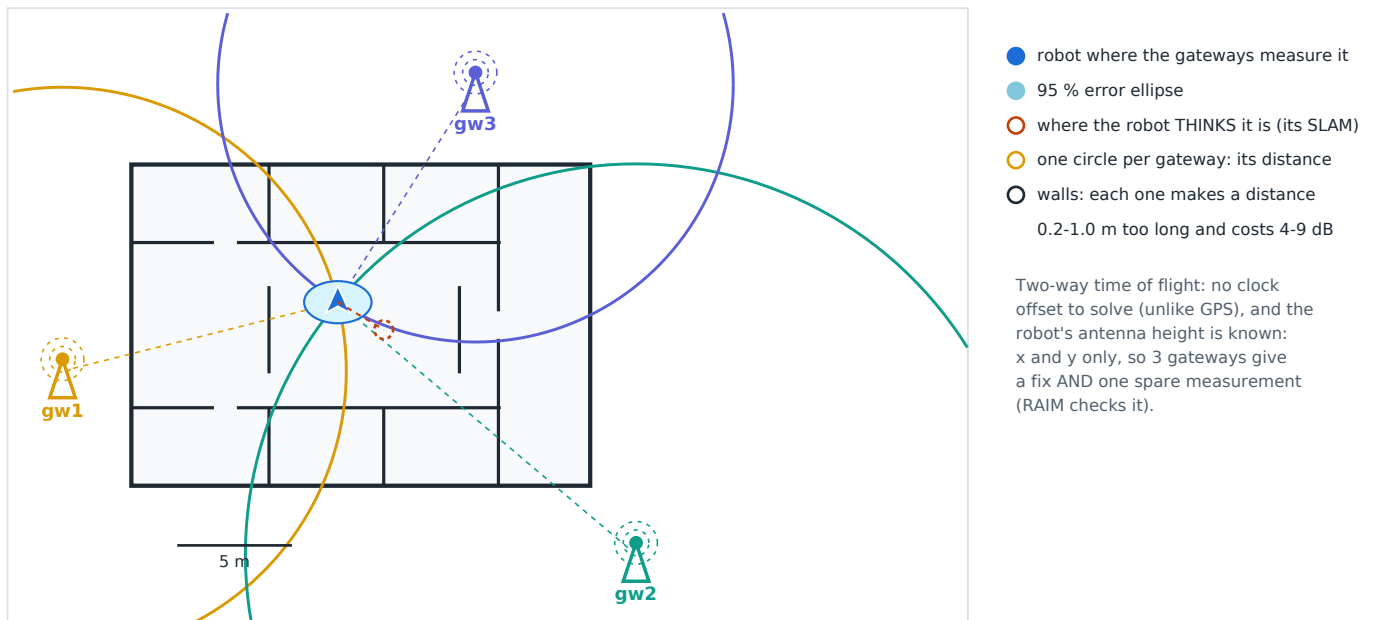


Figure 10. The big Gazebo map with the three gateways of `config/ona_gazebo_big.json`. Each circle is a measured distance; the robot is where they cross.

step	what the ONA does	why
fix	linear first guess, then Gauss-Newton least squares with Huber weights; HDOP from the geometry	robust to one bad range

step	what the ONA does	why
RAIM	chi-square test of the residuals (3 ranges, 2 unknowns: one spare); with 4+ gateways it also excludes the bad one	a fix that does not add up is flagged
walls	each wall costs 4–9 dB and lengthens the distance 0.2–1.0 m; the ONA reads the number of walls from the excess signal loss and shortens the range, with a wider error bar	error 1.07 m → 0.42 m RMS
filter	Kalman filter on the ground plane: predicted by the robot's own displacement, corrected by each range on its own (tight coupling)	tracks on with 1 or 2 gateways
honesty	range noise × 2.5 (the wall error repeats), less weight while parked, × 1.6 with fewer than 3 gateways, no teleporting (> 1 m/s)	the 95 % ellipse holds the robot in 100 % of test pings
cross-check	robot's own pose (SLAM → GPS) vs the measurement: 3σ , 3 pings in a row, and a 3-sphere fix must disagree too → SLAM_DRIFT ; "agrees again" only when its pose really comes back	catches a slipped SLAM, no false alarm

Results (zone simulator, a 22 × 14 m building, every range through 1–4 walls): robot position 0.15–0.88 m RMS, 95 % of pings within 0.97 m, error bars honest in 100 % of pings, no false drift alarm; a 2.9 m SLAM slip caught and the track 0.32 m RMS afterwards; one gateway down for 80 s: worst error 0.55 m. With the Executor's real nodes on the big Gazebo map (2D simulator): 0.66 m RMS while driving.

10.3 GPS for the Command Post: frame translation

geodesy.py converts WGS84 ⇌ ECEF ⇌ local East-North-Up, and LocalToGpsCalibrator fits the map → ENU transform with Umeyama's method: from the anchor and the compass direction of map +x (what the Gazebo configs use), or from 3 or more surveyed entrance markers (a real site). A 2D SLAM map is level, so the fit is planar (only the yaw is unknown): GPS altitude noise cannot tilt the frame. Good fixes are (map, ENU) pairs: the ONA refits the alignment during the mission and compares it with the entrance calibration (not while a SLAM is drifting). The ONA panel shows **holds** or **DRIFTED** (a frame off by more than 1.5 m or 6°, e.g. a wrong north at the entrance).

10.4 Store and forward: LTE, satellite, nothing lost

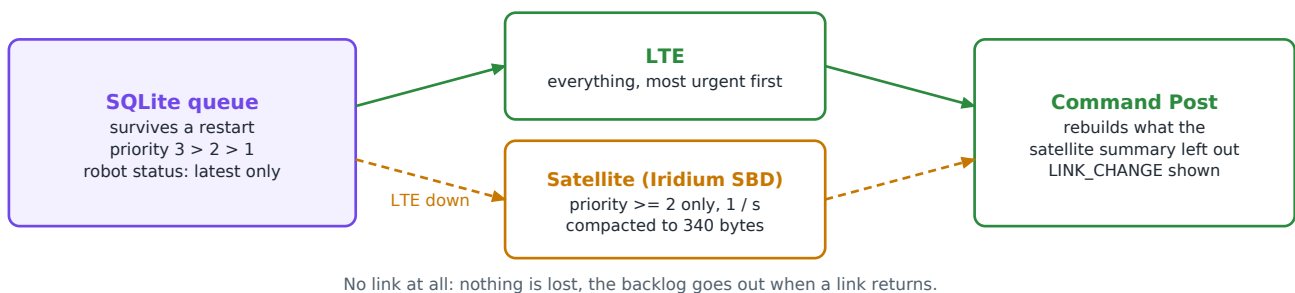


Figure 11. Every message goes through the on-disk queue first.

- **Priorities:** 3 = victims, radiation, gas, fire, alerts; 2 = other hazards, robot positions, lost beacons; 1 = trail beacons, vote counts, ONA status. A robot position replaces its unsent older value.
- **LTE** first. **Satellite** (Iridium SBD model) only when LTE is down: priority ≥ 2, one message a second, each compacted to fit 340 bytes. The Command Post rebuilds what was left out.
- **LINK_CHANGE** is a priority-3 event: the dashboard shows SATELLITE and keeps the gateways green instead of red.
- Every decision (rejected frame, vote, alert, link change, briefing) is written to the `audit` table of the same database: the mission can be replayed afterwards.

10.5 Briefing the Executor

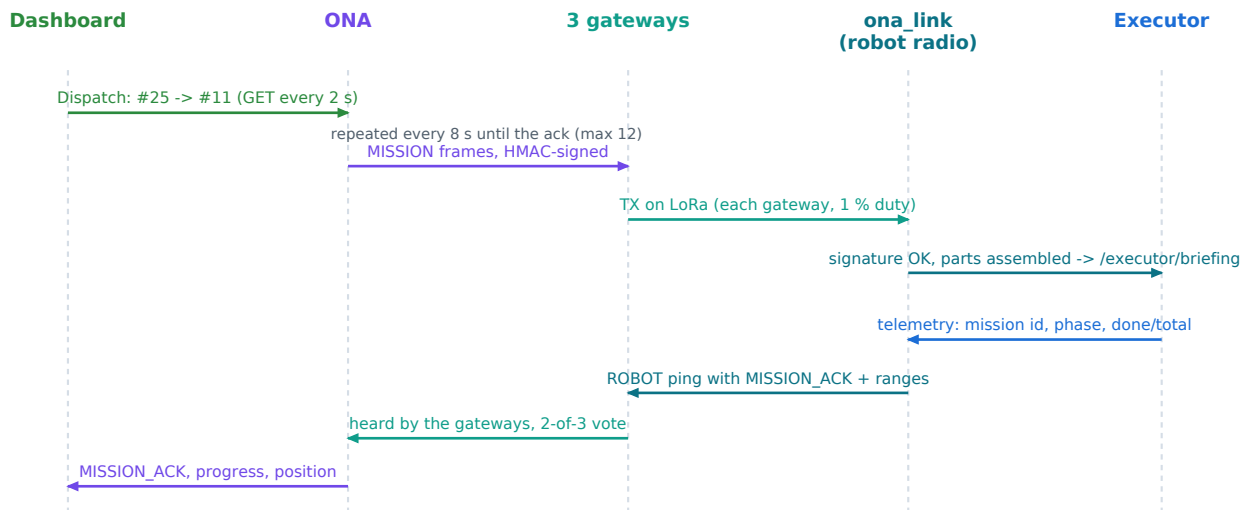


Figure 12. A briefing and its acknowledgement.

The ONA polls GET `/api/mission-dispatch/latest`, signs the mission as MISSION frames (9 targets per frame), and every gateway transmits them, again every 8 s (at most 12 times) until the Executor's pings carry the acknowledgement (**MISSION_ACK**). A signed briefing the ONA did not send is an **UNKNOWN_BRIEFING** alert. What the Executor does with it is in section 11.5.

10.6 The new frames

Same 44 bytes, same header, same MAC rule as a beacon record. The type byte is signed, so one type can never be passed off as another. The beacons drop them as foreign (no relay, no cache, no reject).

offset	ROBOT 0x23 (robot → gateways, every 2 s)	MISSION 0x24 (ONA → gateways → Executor)
0-5	type, network, relay, hops, flags (2-5 unsigned)	same
6-9	robot id, sequence	mission id, version
10-11	role and phase; flags: pose valid, stuck, low battery, briefing ack , emergency	robot id (0xFFFF = any); target count and flags: return to exit, ABORT, in order
12-19	x, y in mm (robot's own map frame)	part i of n; time
20-27	yaw, own position error, odometry, battery, done total	9 target beacon ids
28-35	time, mission id being executed, last beacon passed	(targets continue)
36-43	MAC	MAC

10.7 The robots in Gazebo: ona_link

In the simulation the gateways are virtual. `ona_link_node` runs next to each robot (option `ona`) and sends the ONA, over UDP, exactly the lines three real gateways would print. The ONA itself is the same program as for real gateways.

input	use
TF map → base_footprint	the robot's OWN pose (SLAM): what its ROBOT frame reports
/odom (Gazebo's OdometryPublisher)	the TRUE pose: what the gateways' distances measure
the world file	the walls: each gets its own loss (4-9 dB) and extra distance (0.2-1.0 m), the same at every ping

input	use
/executor/telemetry, /writer/explorer_status	phase, briefing ack, progress
mission file / /writer/mission_record	the beacon records, delivered to each gateway straight or through the beacon nearest to it, repeated now and then
UDP from the ONA	briefing frames: checked (HMAC), assembled, published on /executor/briefing (latched)

It also measures how fast Gazebo runs against the wall clock (often 10-50 % in a VM) and tells the ONA, so that the filter's speed limits are in simulated seconds. Gateway layouts: config/ona_gazebo_big.json (HDOP ≤ 1.54 inside the big building) and config/ona_gazebo_retreat.json (≤ 1.45), read by the robots and by the ONA.

10.8 What the operator sees

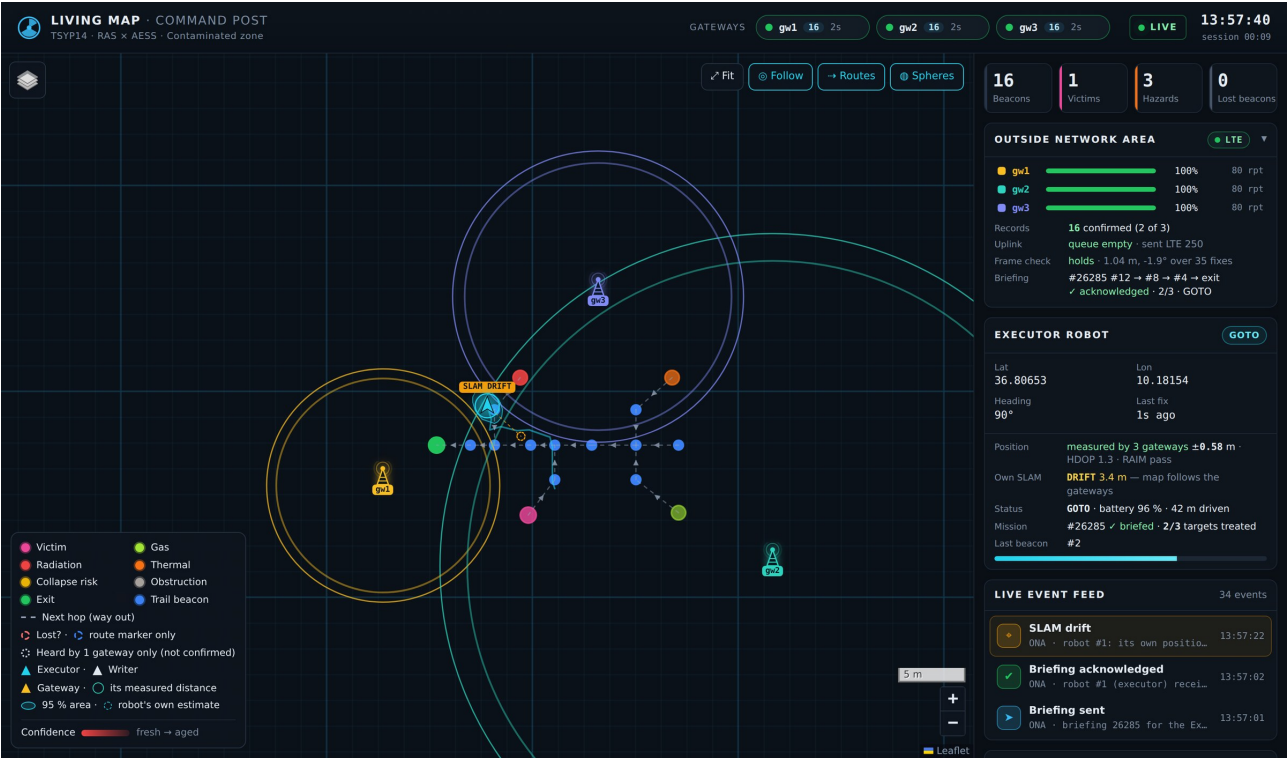


Figure 13. The Executor measured by the three gateways: one circle per gateway, the robot where they cross. Its SLAM has slipped 2.3 m (the small dashed circle): SLAM DRIFT, the map follows the gateways.

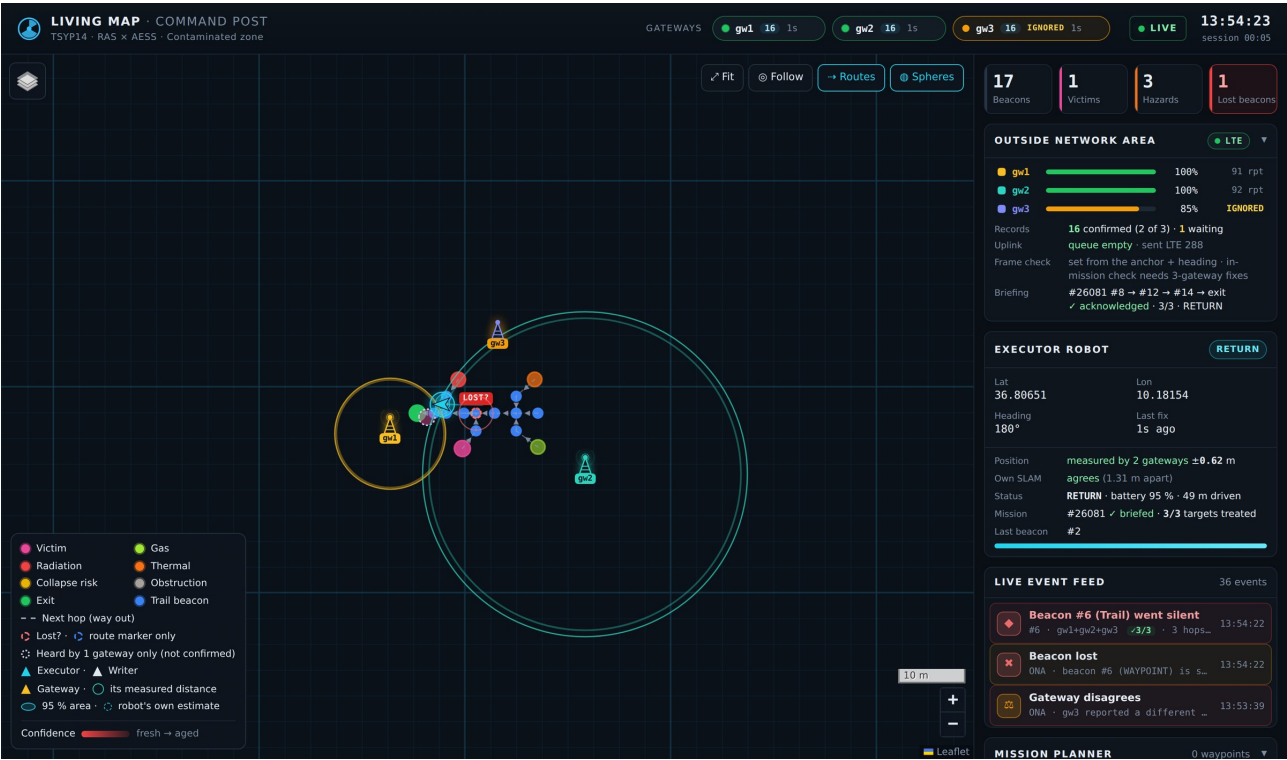


Figure 14. A faulty run: gw3 re-signs changed records and is IGNORED; a forged victim heard by one gateway stays unconfirmed; a crushed beacon is LOST?; the briefing #8 → #12 → #14 is acknowledged and done.

10.9 From the GALERIA code

their file	became	kept	changed or added, and why
trans.py	geodesy.py	WGS84/ECEF/ENU, Umeyama, the calibrator	planar (yaw-only) fit for a flat site; refuses collinear anchors; anchor + heading start; the in-mission check
ekf.py	kalman.py	PositionEKF	RangeEKF: one robust update per range (tight coupling), Joseph form, adaptive noise, error ellipse
ona*.py	store.py, uplink.py, core.py	frame + HMAC idea, de-duplication, SQLite queue, LTE/satellite choice, audit log	priorities and coalescing (the old queue file is upgraded in place), SBD compaction, HTTP to the dashboard; the vote replaces de-duplication; briefings go out as signed LMB2 frames

Their originals are kept unchanged in `ona/galeria_original/`; the tests include their own example and their original queue database. `ona/ONA.md` explains every change in detail.

11. The Executor robot

The Executor enters at the exit with no map and no data from the Writer except the beacons' frames, and the Command Post's briefing. Its brain `executor_core.py` subclasses the explorer, so it reuses the planning grid, pure pursuit, keep-out zones, recovery and the LiDAR safety net; only the choice of where to go is new.

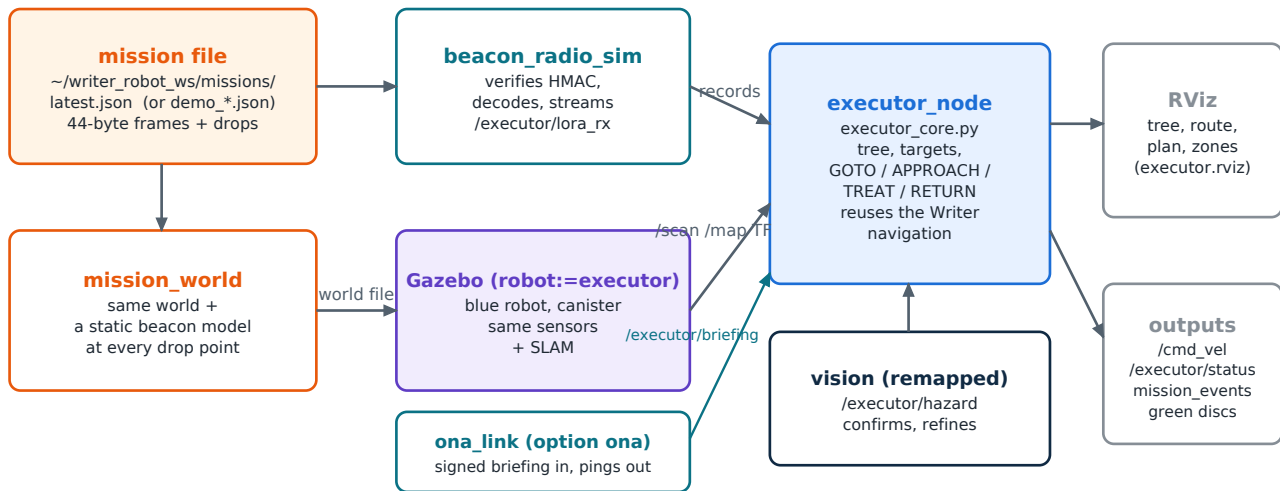


Figure 15. Executor processes (run_executor.sh).

11.1 Receiving the beacons

beacon_radio_sim stands in for a LoRa receiver. Because every beacon holds every record, hearing the EXIT beacon at the entrance gives the whole network. It reads the mission file, verifies each frame's HMAC with the mission key, decodes it and publishes it on `/executor/lora_rx` (JSON: the decoded record, map x/y from the anchor, the frame hex and an RSSI), cycling through all records at 20 Hz over and over, so a late subscriber still gets everything. Frames that fail the check are counted and dropped. With the option `ona`, `ona_link` brings the Command Post's briefings on `/executor/briefing`.

11.2 Rebuilding the tree

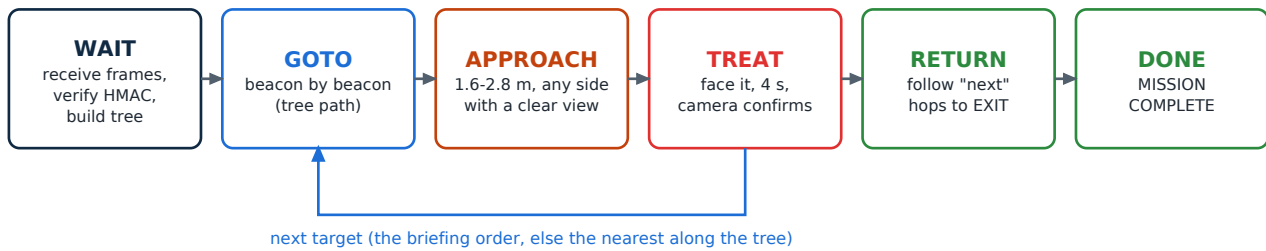
- Each record becomes a node: id, kind, position, next id, distance and bearing.
- For a trail beacon the record position is the beacon. For a hazard beacon it is the hazard; the beacon itself (where the Writer stood, with a clear view) is found from its link: **drop = parent position – next vector** (distance along the compass bearing).
- The tree is rooted at the EXIT; the depth of each node and the chain to the root are cached.
- The mission starts when the EXIT is known and no new record has arrived for 2 s (up to 20 s more while a beacon between a hazard and the exit is still missing). Records arriving later are still added: a hazard whose chain was incomplete becomes a target when the missing beacon arrives, even after MISSION COMPLETE.

11.3 Choosing targets

With a briefing, the targets are exactly the briefed beacons, in the operator's order (section 11.5).

Without one (or after `wait_briefing` seconds without one), they are the records of the kinds in `target_kinds` (RADIATION, THERMAL, GAS, VICTIM), optionally only `target_ids`, not retracted, whose effective confidence is still ≥ 0.15 , and the next target is always the nearest one **along the tree**. With `age_mode: mission` the ages are counted as if the Executor entered right after the Writer finished; `wallclock` uses the real clock. A camera sighting of a hazard that is in no record becomes a keep-out zone (and a retreat if it is in the way).

11.4 Phases



unreachable beacon: skipped · unreachable target: skipped · stuck: RECOVER (max 5 per leg) · unknown hazard seen: keep-out + RETREAT

Figure 16. Executor phases.

phase	what happens
WAIT	receive, verify, decode; wait for the EXIT and 2 s of silence (up to 20 s more while a beacon between a hazard and the exit is missing), and for a briefing if <code>wait_briefing > 0</code> ; build the tree and the target list
GOTO	route = tree path from the current beacon up to the common ancestor, then down to the target's beacon. Each leg (beacon to beacon) is planned on the Executor's own map with unknown space counted as passable (the Writer drove there), with a small extra cost. The robot heads for the next beacon when 0.9 m from the current one. An unreachable beacon is skipped (3 failed legs), a stuck leg gets at most 5 recoveries. A briefed trail or exit beacon is simply visited.
APPROACH	candidate standoff points all around the hazard, ~0.3 m apart on rings of 1.6, 2.0, 2.4 and 2.8 m (victims 0.8–1.2 m), each with a clear view of the hazard on the Executor's map. One Dijkstra from the robot picks the cheapest: path length + 6 m per metre beyond 1.6 m + up to 1 m for leaving the Writer's line of sight. On arrival the hazard must be in sight (map or camera in the last 2 s). A failed point (no path, stuck, no view, > 60 s, or only a > 12 m detour through unmapped space) is dropped and the next one tried, up to 4; a hazard in sight within 3 m is treated from where the robot is. Hazards are 1.2 m keep-out zones throughout.
TREAT	face the hazard, 4 s (<code>treat_time</code>). The camera confirms and refines its position. A green disc is spawned in Gazebo and the zone turns green in RViz; <code>/executor/mission_events</code> reports it.
RETURN	first one more attempt at every skipped hazard, then follow next hops from beacon to beacon to the EXIT
DONE	MISSION COMPLETE: <code>n/n targets treated</code> , stopped at the exit (or, after a "stay" briefing, waiting at its last target for the next one)

11.5 Briefed by the Command Post

`ExecutorCore.set_briefing()` receives a verified briefing `{mission_id, seq, targets, return_to_exit, abort, in_order}` from `/executor/briefing`:

briefing	what the Executor does
targets [25, 11, 20]	exactly these, in this order ; hazards are approached and treated, a trail or exit beacon is visited; a briefed hazard is taken even if its confidence has faded
arrives before the start	used for the whole mission (<code>wait_briefing</code> seconds of waiting: 60 with the <code>ona</code> option, 0 otherwise)
arrives during the mission	re-targets on the spot (a hazard being treated is finished first)
<code>return_to_exit: false</code>	after the last target it stops there and waits for the next briefing
<code>abort: true</code>	stops at once (even mid-treatment) and follows the next hops to the exit
a target with no record yet	taken as soon as its record (or the missing link to the exit) arrives

The robot acknowledges in every ROBOT frame: `/executor/telemetry` (1 Hz: phase, mission id, ack, targets done / total, last beacon) → `ona_link` → the gateways → the ONA → the dashboard ("✓

acknowledged · 2/3 · GOTO").

11.6 Topics and parameters

topic	direction	content
/executor/lora_rx	in	decoded records from the radio
/executor/briefing	in	the Command Post's briefing, signature checked by <code>ona_link</code> (latched)
/executor/hazard	in	the Executor's camera (vision node remapped)
/map, /scan, TF	in	its own SLAM
/cmd_vel	out	drive
/executor/route	out	the beacons still to visit (green line in RViz)
/executor/plan	out	the current leg (orange line)
/executor/markers	out	beacon tree, links, hazard zones
/executor/status	out	phase and progress text
/executor/telemetry	out	1 Hz JSON: phase, mission id, ack, done / total, last beacon (for <code>ona_link</code>)
/executor/mission_events	out	one JSON message per treated hazard

parameter	default	meaning
target_kinds	RADIATION,THERMAL,GAS,VICTIM	
target_ids	(all)	e.g. "12,30": only these beacons
wait_briefing	0	s to wait for a Command Post briefing once the records are in (0: go at once; a briefing still re-targets)
min_confidence	0.15	skip events that have faded below this
age_mode	mission	'mission': enter right after the Writer; 'wallclock'
standoff	1.6	treat hazards from this distance (m)
victim_standoff	0.8	distance to a victim (m)
treat_time	4	s (the demo "treatment": -p treat_time:=6 for longer)
treat_max	3	m: a hazard in plain sight this close can be treated from where
approach_tries	4	standoff points tried before a hazard is skipped
approach_timeout	60	s per standoff point
approach_max_path	12	m: a longer way to a standoff point is a search, not an approach
retry_skipped	true	before going home, one more attempt at every skipped hazard
refine_range	2.6	m: only closer camera sightings move a hazard (the range of a
waypoint_switch	0.9	head for the next beacon when this close to the current one
settle_time	2	s without new records before the mission starts
link_wait	20	s to wait for a missing beacon of a hazard's chain to the exit
max_leg_fail	3	failed legs before a beacon is skipped
unknown_is_free	true	plan through space its map has not seen

parameter	default	meaning
unknown_cost	1.5	extra cost of unknown cells
coverage	false	no exploration: the beacons say where to go
(all explorer parameters)		e.g. keepout_radius 1.2, v_max 0.70, robot_radius 0.36

11.7 Beacon-graph navigation (ready, not wired in yet)

writer_robot/beacon_graph_nav.py turns the beacons into a weighted graph and finds routes with Dijkstra or A*. The Executor still follows the tree; the module is written and tested so that it can replace the tree routes later, in five small steps listed at the end of the file.

part	what it is
nodes	every beacon, at its drop point (rebuilt from the frames exactly like the Executor does)
tree edges	beacon → its next: always kept, the Writer drove them
shortcut edges	two beacons closer than link_radius (4 m) whose straight line is free on an occupancy grid (the Executor's own SLAM map, or the world walls in the tools); weight = length × 1.15
hazards	an edge crossing a keep-out disc is removed; one passing within 0.5 m of it costs 3 m more
searches	dijkstra(a, b), astar(a, b) (straight-line heuristic, admissible), tree_path(a, b) (today's behaviour), shortest_from(a), order_targets(start, targets, end) (nearest neighbour + 2-opt)

On the big demo mission (43 beacons) the graph adds 46 wall-free shortcuts. From the EXIT, the routes to the four hazards get 1.6–4.9 m shorter (gas: 26.8 → 21.9 m), visiting all four and returning costs 57.6 m instead of 67.2 m, and A* expands about half as many beacons as Dijkstra for the same routes.

12. Simulation and verification

Gazebo cannot run in the build environment, so the robot logic is verified in two ways that need no ROS: a 2D simulator of the same worlds, and the real ROS node files running on a small stand-in for rclpy. The simulated SLAM behaves like slam_toolbox (a scan is added only after 0.3 m or 0.3 rad of motion, a cell is known only after more than 2 rays crossed it). Every test below also runs on Windows.

test	what it checks	result
tools/sim2d.py	the Writer in 2D: LiDAR, growing map, camera with FOV, blob size and line of sight, footprint, acceleration limits	big map explored in ≈ 7 simulated min, 4/4 hazards found, back home
tools/run_tests.py	21 Writer runs: 3 buildings and the Gafsa mine, several seeds, cameras seeing later, LiDAR self-hits; no collision, ≥ 0.9 m from hazards, never stalled > 20 s, complete, home; in the mine every resource graded and every area searched	21 / 21
tools/sim_mission.py	the whole mission: the Writer drops beacons (56 on the big map), the Executor follows them; options for briefings	4/4 hazards treated, ≈ 75 m, ≈ 3 min
tools/run_mission_tests.py	21 missions: 3 maps, seeds, late camera, radio joining mid-cycle, Executor map misaligned 0.5 m / 3°, a camera 25 % off in range, beacons heard late, a door narrowed to 0.6 m and a blocked door, 4 briefings (in order, re-target and stay, abort, briefed record heard late), and 5 in the Gafsa mine (victims first, a sampling briefing, ±15 % sensor range errors)	26 / 26
tools/test_nodes.py	the ROS nodes themselves: vision on rendered RGB-D frames, explorer, beacon node, recorder, radio and Executor driving the 2D simulator through the ROS topics, the mine sensors node	70 / 70

test	what it checks	result
tools/test_beacon_graph.py	beacon graph: A* = Dijkstra, A* expands fewer nodes, shortcuts wall-free and outside keep-outs, target ordering	21 / 21
tools/test_ona_link.py	the robots and the real ONA together: same frame codec, the gateways' radio in both worlds, briefing obeyed in order, 43/43 records confirmed 3/3, the Executor measured through the walls, a SLAM slip caught	18 / 18
ona/tests/test_ona.py	frames, geodesy (GALERIA's example), both filters, spheres/HDOP/RAIM, vote, queue (their database), LTE/satellite, end to end, bpsim --gateways 3 through the vote	70 / 70
ona/tests/test_zone.py	whole missions with faults: lying gateway, forged victim, crushed beacon, gateway down, SLAM slip, corrupted frames, strict mode	20 / 20
ona/tests/test_vs_beacon_net.py	600 random records byte-identical to the beacons' codec	3 / 3
beacon_net: make test, make matrix	C $\rightleftharpoons$ Python $\rightleftharpoons$ firmware cross-checks, 42 network scenarios, the gateway firmware compiled for the PC (raw lines, TX)	all pass, 42 / 42

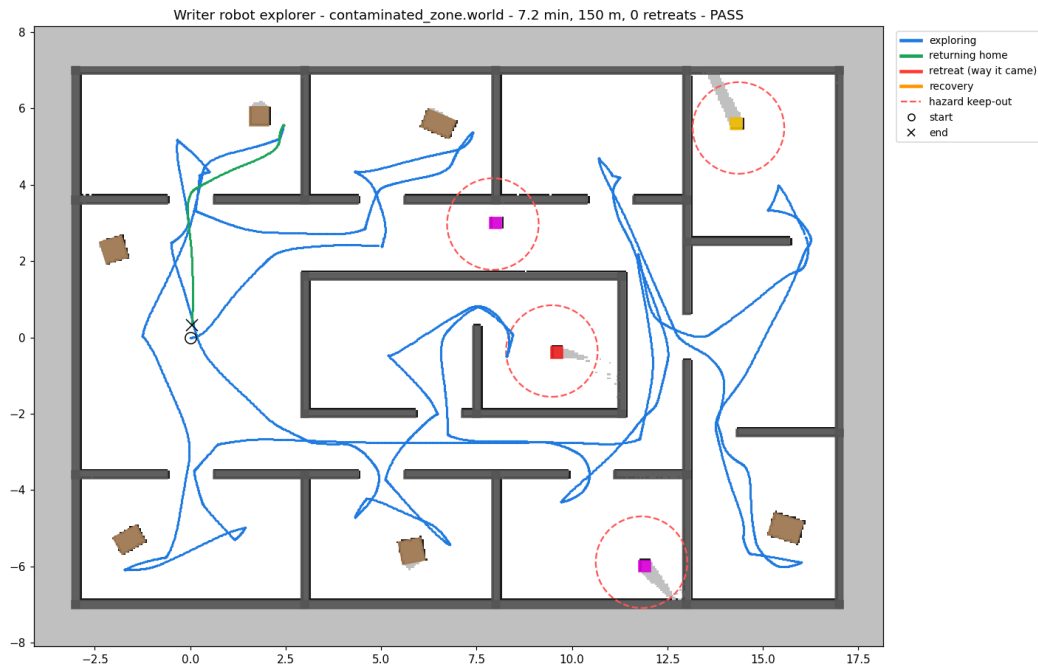


Figure 17. Writer on the big map (2D simulator): path, beacons, hazards and keep-out zones.

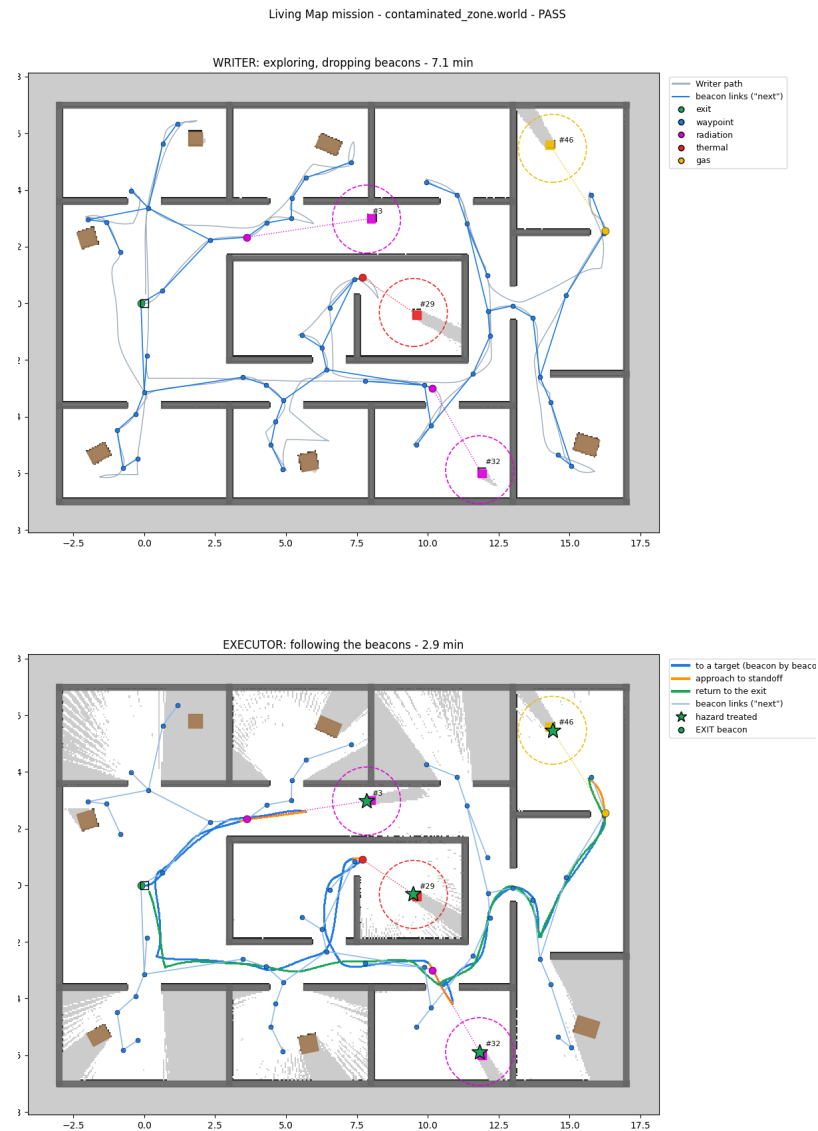


Figure 18. Whole mission: the Writer's run (top) and the Executor following the beacon tree (bottom).

WATCH OUT What these tests cannot prove: how Gazebo renders the block colours in your VM, how slam_toolbox behaves on your CPU, real RF and real ranging through rubble. The camera health log (section 6.2), check_sim.sh and the ONA's honest error bars are aimed at exactly those gaps.

13. Data contracts

Every message between the parts is plain JSON. These are the shapes to expect when you `ros2 topic echo` or read a log.

`/writer/hazard` (**vision** → **explorer, every sighting**)

```
{"type": "fire", "x": 6.21, "y": 3.05, "range": 2.4, "bearing": 0.112, "area": 5200, "stamp": 311.4}
```

`/writer/events` (**vision** → **beacon node, once per hazard**)

```
{
  "event_type": "fire", "value": 0.13, "range_m": 2.4, "bearing_rad": 0.112,
  "x": 3.9, "y": 2.8, "theta": 0.1,           <- robot pose = where the beacon is dropped
  "event_x": 6.21, "event_y": 3.05,          <- the hazard itself
  "timestamp": 1790000311.2, "source": "vision"}
}
```

/executor/lora_rx (radio → Executor)

```
{
  "beacon_id": 13, "seq": 1, "kind": "RADIATION", "gps": {"lat": ..., "lon": ..., "err_m": 1.4, "src": "slam"},
  "severity": 22, "confidence": 0.9, "next": {"id": 12, "dist_m": 2.1, "bearing_deg": 181.0},
  "ts": 1790000400, "ttl_s": 43200, "half_life_s": 21600, "retracted": false, "stale": false,
  "x": 8.52, "y": 4.10, "anchor": {...}, "frame": "212a0d00...", "rssi": -96}
}
```

/executor/briefing (ona_link → Executor)

```
{
  "mission_id": 26149, "seq": 1, "targets": [12, 8, 4], "return_to_exit": true, "abort": false,
  "in_order": true, "ts": 1790863137}
}
```

A gateway line with ranging (gateway or ona_link → ONA)

```
{
  "gw": "gw2", "ev": "rx", "frame": "232a0100...", "rssi": -96.4, "snr": 10.0,
  "range_m": 27.81, "range_sd": 0.5, "range_src": "tof"}
}
```

Dashboard POST /api/beacon (ONA or bridge → dashboard)

```
{
  "beacon_id": 8, "event_type": "victim", "value": 61, "lat": 36.8064531, "lon": 10.1815762,
  "timestamp": 1789999560, "ttl": 16, "gateway_id": "gw1+gw2", "rssi": -107.3, <- original contract
  "v": 2, "ev": "update", "kind": "VICTIM", "seq": 1, "severity": 61, "confidence": 0.902,
  "eff_conf": 0.825, "err_m": 0.6, "gps_src": "slam", "next": {"id": 7, "dist_m": 1.2, "bearing_deg": 0.0},
  "ttl_s": 7200, "half_life_s": 3600, "age_s": 463, "hops": 5, "relay": 0, "snr": -0.5,
  "silent": false, "stale": false, "retracted": false,
  "ona": {"state": "confirmed", "votes": 2, "of": 3, "quorum": 2, "gateways": ["gw1", "gw2"]}}
}
```

ttl is 20 × the effective confidence, so an old dashboard still fades pins correctly.

Dashboard POST /api/robot-status (ONA → dashboard, excerpt)

```
{
  "robot_id": 1, "role": "EXECUTOR", "phase": "GOTO", "lat": 36.8065098, "lon": 10.1814979,
  "sigma_m": 0.48, "source": "gateways", "votes": 3, "of": 3,
  "measured": {"lat": ..., "lon": ..., "sigma_m": 0.48, "ellipse": {"a_m": 0.48, "b_m": 0.35, "angle_rad": 1.69}},
  "fix": {"hdop": 1.2, "raim": "pass"}, "reported": {"lat": ..., "lon": ..., "x": -0.03, "y": 0.01},
  "consistency": {"state": "agree", "dist_m": 1.09, "sigmas": 2.06},
  "ranges": [{"gw": "gw1", "range_m": 5.44, "raw_m": 6.16, "walls": 1.2, "horiz_m": 5.17, "sigma_m": 0.71}, ...],
  "mission_id": 15292, "mission_ack": true, "done": 2, "total": 3, "battery_pct": 93}
}
```

14. The Gafsa mine scenario

The whole chain also runs underground, in a simulated room-and-pillar phosphate panel of the Gafsa basin (run_explorer.sh mine). *3_Gafsa_Mine_Case.pdf* covers the dangers, the sensors, the robot specification and the radio; here is where it lives in the software.

piece	what it adds
tools/make_mine.py	one layout -> the Gazebo world worlds/gafsa_mine.world, the site file worlds/gafsa_mine.json (dangers, resources, search areas, thresholds), the dashboard plan command_post/public/sites/gafsa_mine.geojson, the mine ONA configs
writer_robot/mine_site.py	the instrument models (no ROS): multi-gas and radon spreading along the galleries (walking distance), gamma, thermal camera, roof scanner, XRF wall probe, camera + XRF, the victim search coverage per area

piece	what it adds
mine_sensors node	replaces the vision node underground: <code>/writer/hazard</code> , <code>/writer/events</code> as before, <code>/writer/air</code> (1 Hz); the Executor runs it remapped to <code>/executor/*</code>
URDF mine:=true	the mine sensor head (thermal camera, roof scanner, gas inlet, radon / gamma box, XRF probe, headlights), automatic in the Gafsa world
LMB2 kinds 8-11	PHOSPHATE (severity = % $P_2O_5 \times 4$), GOLD, GEMSTONE, SEARCHED (severity = % of the area seen); marked, never avoided
explorer / Executor	resources and searched areas are never keep-out zones; the Executor treats unstable roofs too, reaches victims first (<code>priority_kinds</code>), samples resources only when briefed, says its tasks the mine way (ventilation, roof prop, oxygen)
ONA	the site in its status, RESOURCE_MARKED and AREA_SEARCHED events, resources last (never over the satellite), a rock radio model for the robots (8 dB/m, time of flight only in line of sight, no RSSI ranges)
Command Post	the mine plan as the map, mine words, measured values decoded from the severity, the Gafsa mine panel (victim search by area, dangers, resources), Resources and Search layers, <code>npm run demo:mine</code>

15. Limitations and next steps

- **The robots' radio on real hardware.** The Executor needs a LoRa receiver that publishes decoded records on `/executor/lora_rx` and checked briefings on `/executor/briefing`, and both robots need a radio that sends the ROBOT frames. The formats are fixed (section 13; `ona_link_node.py` shows the whole logic); the small bridges are not written yet.
- **Ranging hardware.** Positioning by the gateways needs a time-of-flight radio (SX1280 or UWB) on each gateway and robot. With RSSI only, the vote and the uplink still work, but the position is coarse (about ± 45 % of the distance).
- **The accuracy numbers are simulation numbers.** The wall model is standard but still a model: measure the loss and the extra distance per wall on site, and RSSI and delivery through real debris for `bpsim --rubble`.
- **Three gateways give one spare measurement.** RAIM detects a bad range but cannot say which; a 4th gateway would.
- **Vision** uses colour masks tuned to the simulated blocks. Real hazards need a trained detector and real sensors (dosimeter, gas sensor, thermal camera) feeding the same `/writer/events` message.
- **Odometry.** The simulation uses Gazebo's true motion for `odom→base_footprint`. A real robot needs the wheel + IMU EKF (`ekf` option) and SLAM scan matching (`slam_toolbox_params_ekf.yaml`), tuned on the real robot.
- **Shared mission key (LMB2):** a captured beacon exposes it. Per-mission keys and flash encryption mitigate it. The vote limits what one captured gateway can do, not what a stolen key can do everywhere.
- **Network partitions** need a physical fix (a replacement beacon or a data mule).